한국마이스터고_1주차

류영표 강사
ryp1662@gmail.com



Youngpyo Ryu. All rights reserved.
Unauthorized reproduction or distribution is prohibited.

류 영 표

AI & Applied Mathematics Researcher · Lecturer

ryp1662@gmail.com



학력

- 동국대학교 일반대학원 응용수학 석사 수료 (2016-2019)
- 동국대학교 수학과 졸업 (2010-2016)

심사 및 전문가 자문 활동

- 인공지능 학습용 데이터 구축 사업 품질검증 전문가 자문위원 (2023)
- 2023 신뢰할 수 있는 인공지능 개발 안내서(안)_고도화 전문위원
- 2023 자율주행/의료/공공사회 분야 신뢰할 수 있는 인공지능 개발 안내서(안) 자문위원

연구 및 프로젝트

- 부산대학교 병원: 개인 맞춤형 당뇨병 예방·관리 인공지능 시스템 외주 프로젝트 (2022)
- 연세대학교 병원: 영상의학과 FLASK 고도화 연구 (2023, 2024)
- 산업수학 스터디 그룹 (국가수리과학연구소): 피부암/유방암 분류, 유전자 정보 분석
- 폐플라스틱 이미지 객체 검출 경진대회 3위 (2021)
- 동국대학교 수학과 연구원(2025.10 ~ / 양자 AI, XAI 연구 진행 중)

논문

- Quantum computing for the optimization of CT image reconstruction, IEEE ICTC 2022

㈜ 헬로월드랩스 전담교육팀 과장

강의 및 교육 활동

대학 강의

- 고려대학교, 인천대학교, 동양미래대학교, 목포대학교
- 이젠 아카데미: 파이썬/ADSP 과정

기업 교육

- 현대자동차 연구원
- 한화, 하나금융, 한전 KDN, IBK 기업은행, 아가방앤컴퍼니, IM증권

공공/국비 과정

- [Est Camp] AI 퀀트 캠프 1기
- 보건·바이오 AI 활용 전문가 양성과정 (1-3기)
- 국제인공지능올림피아드 한국 대표 멘토
- 충청 ICT 취창업 역량강화 프로그램 (SQLD)

아카데미 & 부트캠프

- 모두의연구소 Aiffel 퍼실리테이터 (1기)
- Upstage AI Boostcamp 멘토 (1-6기)
- 패스트캠퍼스/멀티캠퍼스 조교 및 멘토
- 공개SW개발자대회 멘토

전체 교육 목표

90분의 제한 시간 안에
데이터 분석 문제를 스스로 해결하고
제출까지 완료하는 **실전 역량**을 기릅니다.



교육의 최종 목표

수강생이 90분이라는 제한된 시간 안에 데이터 분석 문제를 스스로 수행하고 제출까지 완료할 수 있도록 합니다.

데이터 분석 문제 해결 프로세스

실제 대회와 동일한 흐름으로, 처음부터 끝까지 스스로 수행할 수 있도록 연습합니다.

1. **문제 이해** — 문제 유형, 평가지표, 제출 형식 확인
2. **데이터 진단** — 분포, 결측, 이상치, 변수 관계, 분포 이동
3. **검증 설계** — 적절한 분할 전략, 누수 방지 설계
4. **Baseline** — 기준선 모델로 현재 수준 파악
5. **모델링** — 적절한 모델과 특성 설계
6. **성능 평가** — 적절한 지표로 성능 분석 및 해석
7. **개선** — 통계적으로 의미있는 개선 여부 판단
8. **제출** — 최종 예측 및 제출 파일 생성

이 과정에서 기르는 핵심 역량



기준선 비교와 점수 해석

모델 점수를 기준선과 함께 올바로게 해석할 수 있습니다.



p-value와 효과 크기 구분

통계적으로 차이가 있는지와 실무적으로 의미가 있는지를 구분해서 판단할 수 있습니다.



통계적 개선 여부 판단

개선 폭이 단순한 분할 잡음인지, 실제 개선인지 결정할 수 있습니다.



데이터 누수 방지

목표변수가 포함되는 모든 계산을 학습 데이터 내부에서만 수행하여 누수를 방지할 수 있습니다.

인공지능, 기계학습, 딥러닝 개념

인공지능 (AI)

컴퓨터가 사람처럼 생각하고,
판단하게 만드는 기술



머신러닝 (ML)

인공지능의 한 분야

인간의 학습능력과 같은 기능을
컴퓨터에 부여하기 위한 기술



딥러닝 (DL)

**인공지능 신경망을 기반으로 한
머신러닝 방법론 중 하나**

빅데이터를 기반으로 스스로 학습하며,
판단하는 기술



1차 AI 붐
탐색·추론

2차 AI 붐
전문가 시스템

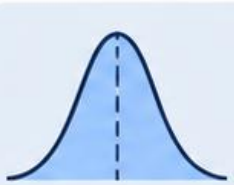
3차 AI 붐
기계학습·특징표현학습

1950's 1960's 1970's 1980's 1990's 2000's 2010's

통계 VS 머신러닝

통계

분포나 가정을 사용해서
엄격한 규칙이 적용되는 설문조사나 실험 계획에 사용됨



가정 기반 분석

데이터가 특정한 분포(예: 정규분포)를
따른다는 가정을 사용



엄격한 규칙과 해석

통계적 유의성, 신뢰구간 등
명확한 규칙에 따라 분석하고 해석



설문조사 · 실험 계획에 주로 사용

A/B 테스트, 임상실험, 설문조사 등
잘 설계된 실험에서 효과를 검증

적은 데이터에서도
원인과 관계를 설명하는 데 강점이 있습니다.

VS

머신러닝

대용량 데이터의
분석이나 패턴을 찾는데 사용됨



대용량 데이터 활용

많은 양의 데이터를 학습하여
데이터 속에 숨겨진 패턴을 발견



패턴 중심 예측

명시적인 가정 없이
데이터로부터 자동으로 패턴을 학습



다양한 분야에 활용

이미지, 음성, 텍스트, 추천 시스템 등
복잡한 문제의 예측과 분류에 사용

많은 데이터에서
복잡한 패턴을 찾아 예측하는 데 강점이 있습니다.

머신러닝 VS 데이터 마이닝

머신러닝

훈련 데이터를 통해 **학습된**
알려진 속성을 기반으로 한 **예측**으로 중점



데이터로부터 학습

훈련 데이터를 사용하여
패턴을 학습하고 모델을 생성



예측이 주요 목적

알려진 속성을 기반으로
새로운 데이터에 대해 **예측**



대표적인 방법

회귀, 분류, 신경망 등
데이터 기반의 예측 모델

공통점

데이터에서
유의미한 패턴을
찾아내는 것

데이터마이닝

데이터의 미처 몰랐던 속성을 **발견**하는 것에
집중. 데이터베이스의 지식 발견 과정에 해당



숨겨진 패턴 발견

대용량 데이터에서 미처 몰랐던
흥미로운 패턴, 규칙, 연관관계를 발견



탐색과 분석이 주요 목적

데이터 속의 새로운 지식과
패턴을 찾아내어 이해



대표적인 방법

연관 규칙 분석, 군집 분석,
이상치 탐지, 패턴 탐색 등



이들은 **중복되는 부분이 있다.** 데이터 마이닝에서 머신러닝은 필수가 아니지만
머신러닝에서는 데이터마이닝 필수라는 부분이 다르다.

머신러닝의 정의

> A computer program is said to **learn from experience E** with respect to some class of tasks **T** and performance measure **P**, if its performance at tasks in T, as measured by P, **improves with experience E.**
>
> – TOM M. MITCHELL, Carnegie mellon University

E 경험 (Experience)



학습에 사용되는
데이터, 또는
데이터로부터 얻는 경험

T 과제 (Tasks)



해결하고자 하는
문제의 종류
(예: 분류, 회귀 등)

P 성능 측정 (Performance)



모델의 성능을
평가하는 기준
(예: 정확도, 오차 등)

성능 향상 (Improves)



경험이 중가할수록
과제 T에서의 성능 P가
점진적으로 향상됨



1. 테스크(T)에 대해 꾸준한 **경험(E)**을 통해 T에 대한 성능(P)을 높이는 것을 **기계학습**이라고 한다.
2. 기계학습에서 가장 중요한 것은 **E에 해당하는 데이터**이다.
3. **좋은 품질의 데이터**륵 많이 가지고 있다면 보다 높은 성능을 끌어 낼 수 있다.

Machine learning process

1 현실 세계

해결하고자 하는 문제 상황



2 데이터 수집

문제와 관련된 데이터를 모음



3 특징 공학 (Feature Engineering)

데이터를 모델이 학습할 수 있는 형태로 변환

데이터 전처리

결측치 처리, 스케일링, 인코딩 등

특징 추출

데이터에서 유의미한 정보를 추출

특징 선택

중요한 특징만 선택

4 모델링 (Modeling)

데이터로부터 패턴을 학습하는 모델을 생성

모델 구축

알고리즘 선택 및 모델 학습

모델 평가

검증 데이터를 사용하여 성능을 평가

모델 최적화

하이퍼파라미터 튜닝, 성능 개선

5 최종 모델

최적화된 모델을 완성



6 신규 데이터

새로운 데이터에 적용하여 예측 수행



7 특징 공학

학습 때와 동일한 전처리, 특정 추출 및 선택을 적용

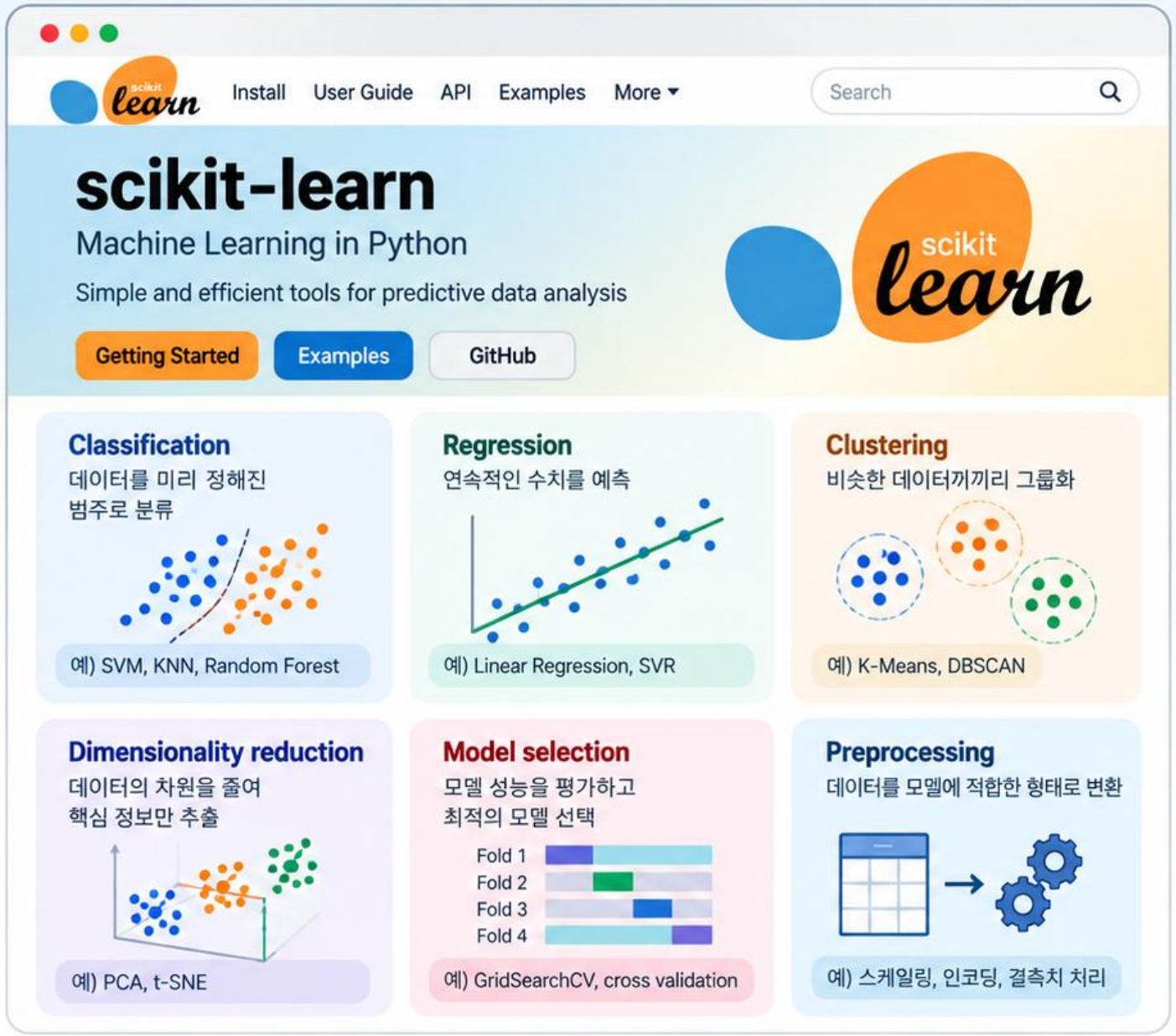


8 판단 결과

예측 결과를 도출하여 문제 해결에 활용



Scikit-learn 이란?



1. 파이썬 기반의 다른 머신러닝 패키지도 사이킷런 스타일의 API를 지향할 정도로 **쉽고 파이썬스러운 API**를 제공한다.

2. 머신러닝을 위한 매우 **다양한 알고리즘**과 개발을 위한 **편리한 프레임워크**와 **API**를 제공한다.

3. 오랜 기간 **실전 환경에서 검증**되었으며, 매우 많은 환경에서 **성숙한 라이브러리**이다.

4. 주로 **NumPy와 SciPy 기반** 위에서 구축된 라이브러리로, **데이터 분석 생태계와 매끄럽게 연동**된다.

Scikit-learn 이란?

문제 상황에 맞는 알고리즘을 쉽게 사용할 수 있는 파이썬 머신러닝 라이브러리

START

더 많은 데이터가 있나요?

- 예 → > 50 샘플
- 아니요

예측하려는 값은 무엇인가요?

- 범주 (카테고리)
- 연속값 (수치)

데이터에 레이블이 있나요?

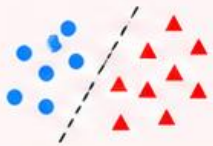
- 예
- 아니요

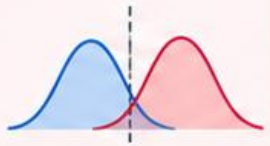
데이터 구조가 복잡한가요?

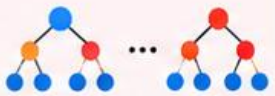
- 아니요 → 단순한 구조: 선형 모델, 트리 기반 모델 등을 우선 사용
- 예 → 복잡한 구조: 비선형 모델, 앙상블 모델 등을 고려

분류 (Classification)

카테고리 값을 예측하는 문제

- **SVC** — 서포트 벡터 머신

- **KNN** — k-최근접 이웃

- **Naive Bayes** — 나이브 베이즈

- **Decision Tree** — 의사결정나무

- **Random Forest** — 랜덤 포레스트

- **Logistic Regression** — 로지스틱 회귀


회귀 (Regression)

연속적인 수치를 예측하는 문제

- **Linear Regression** — 선형 회귀

- **Ridge / Lasso** — 규제 선형 회귀

- **Decision Tree** — 의사결정나무

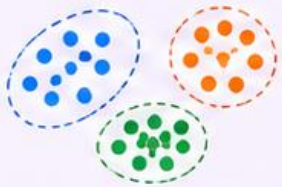
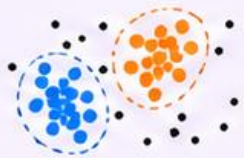
- **Random Forest** — 랜덤 포레스트

- **SVR** — 서포트 벡터 회귀

- **Gradient Boosting** — 그래디언트 부스팅

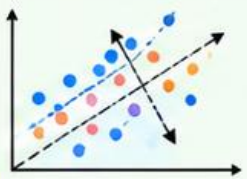

군집화 (Clustering)

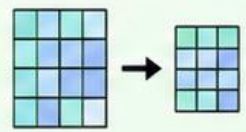
비슷한 데이터를 그룹으로 묶는 문제

- **KMeans** — K-평균 군집화

- **DBSCAN** — 밀도 기반 군집화

- **Hierarchical** — 계층적 군집화


차원 축소 (Dimensionality Reduction)

데이터의 차원을 줄이는 문제

- **PCA** — 주성분 분석

- **t-SNE** — 비선형 차원 축소

- **Truncated SVD** — 희소 행렬 차원 축소


1차시 (60분)

문제 정의와 Baseline 만들기

데이터 분석 문제를 정확하게 읽고, Baseline부터 첫 제출까지 완주하기

2026 MAICON
데이터 사이언스 교육
예선 실습 7시간 + 본선 준비 1시간

차시 개요

주제

데이터 분석 문제를 정확하게 읽고, Baseline부터 첫 제출까지 완주하기

시간

60분

핵심 내용

문제 이해 → 데이터 확인 → Baseline → 모델 비교 → 첫 제출까지 전체 흐름 실습

핵심 질문

대회 문제를 처음 받았을 때 반드시 스스로 답할 수 있어야 하는 질문입니다.

1. **무엇을 예측**해야 하는가?
2. 무엇으로 **평가**받는가?
3. **최소한 어느 정도 점수**는 나와야 하는가?
4. 내가 만든 결과를 **제대로 제출**할 수 있는가?

학습 목표

1차시가 끝났을 때, 다음을 할 수 있습니다.

- Train / Test / Sample Submission의 역할을 설명할 수 있다.
- Target과 ID column을 구분할 수 있다.
- Classification과 Regression을 판단할 수 있다.
- 평가 지표가 요구하는 **예측값의 형태**를 판단할 수 있다.
- **Dummy baseline**의 필요성을 설명할 수 있다.
- **동일한 검증 조건에서 여러 모델을 비교**할 수 있다.
- Test 예측 결과를 submission.csv로 저장하고 다시 김사할 수 있다.

이번 차시의 전체 흐름

문제를 이해하고, Baseline부터 첫 제출까지 한 번에 완주합니다.



오늘의 목표

오늘의 목표는 좋은 모델을 만드는 것이 아니라,
처음 보는 문제를 정확하게 읽고 첫 제출까지 완주하는 것입니다.

Q

대회에서 데이터를 받았습니다.
여러분은 가장 먼저 무엇을 확인하시겠습니까?



- head()를 본다.
- 모델을 선택한다.
- 결측치를 본다.
- Random Forest를 돌린다.

하지만!

모델을 고민하기 전에,

먼저
문제 정의를 해야 합니다.

문제 정의와 Baseline 만들기

데이터 경진대회의 기본 구조

1. 학습 및 검증 (Train 데이터 사용)

Train Data



- Features (X)
- Target (y)

Model Training



전처리 + 모델 학습
(Training)

Validation



검증 점수 확인
(Validation Score)

2. 최종 예측 (Test 데이터 사용)

Test Data



- Features (X)
- Target 없음

Prediction



최종 예측 생성
(Prediction)

Submission



submission.csv
생성 및 제출

1 Train Data

모델을 학습하기 위한 데이터입니다.

ID	temperature	pressure	vibration	target
A001	23.5	1.02	0.18	0
A002	21.1	0.98	0.24	1
A003	25.0	1.10	0.16	0
A004	22.3	1.05	0.20	1
...	...	...	...	...

Train에는 일반적으로 Target이 있습니다.

2 Test Data

최종 예측 대상 데이터입니다.

ID	temperature	pressure	vibration
B001	24.1	1.01	0.19
B002	20.8	0.97	0.22
B003	23.6	1.08	0.17
B004	21.9	1.00	0.25
...	...	...	...

Test에는 Target이 없습니다.
우리가 예측한 값을 제출해야 합니다.

3 Sample Submission

제출 파일의 형식을 알려주는 템플릿입니다.

ID	target
B001	0
B002	0
B003	0
B004	0
...	...

Sample의 0 값은 정답이 아닙니다.
제출 파일의 모양을 보여주는 placoholder입니다.

문제 정의와 Baseline 만들기

문제를 받으면 반드시 적어야 할 다섯 줄

문제 정의 템플릿

데이터를 확인한 후, 아래 다섯 줄을 먼저 작성합니다.

1	Target	:
2	ID Column	:
3	Problem Type	:
4	Metric	:
5	Submission	:

작성 예시

실제 대회 문제를 읽고 다음과 같이 작성할 수 있습니다.

1	Target	: failure
2	ID Column	: equipment_id
3	Problem Type	: Binary Classification
4	Metric	: F1 Score
5	Submission	: id, failure

각 항목을 확인하는 방법

1 Target

- 무엇을 예측해야 하는 값인가?
- train 데이터에만 존재
- 분류: 범주형 / 회귀: 연속형

2 ID Column

- 각 행을 식별하는 고유값
- 보통 예측에는 사용하지 않음
- 제출 파일의 첫 번째 열

3 Problem Type

- Classification (범주형)
- Regression (연속형)
- 값의 의미로 판단
  (데이터 타입뿐만 보지 않기)

4 Metric

- 여면 기준으로 평가받는가?
- 분류: Accuracy, F1, ROC-AUC 등
- 회귀: MAE, RMSE, R^2 등

5 Submission

- 제출 파일의 형식
- 포함되어야 하는 열 이름과 순서
- 분류: 클래스 또는 확률
- 회귀: 연속값

문제 정의와 Baseline 만들기

Target과 ID는 완전히 다르다

1 Target (예측하고 싶은 값)

우리가 예측하고자 하는 값으로, 문제마다 의미가 다릅니다.

Target의 예시

고장 여부	암 여부	고객 이탈 여부	판매량	전력 소비량
(0 / 1)	(정상 / 암)	(이탈 / 유지)	(연속값)	(연속값)

데이터 예시 (분류 문제)

id	temperature	pressure	vibration	target
A001	23.5	1.02	0.18	0
A002	21.1	0.98	0.24	1
A003	25.0	1.10	0.16	0
A004	22.3	1.05	0.20	1
...	...	...	...	...

Python 코드

```
# Target 변수 y 추출
y = train["target"]
```

2 ID (각 행을 식별하는 값)

각 데이터를 식별하기 위한 값으로, 보통 모델의 입력 변수로 사용하지 않습니다.

ID의 예시

patient_id	device_id	customer_id	record_id	id
(환자 식별)	(장비 식별)	(고객 식별)	(기록 식별)	(단순 일련번호)

데이터 예시

id	temperature	pressure	vibration	target
A001	23.5	1.02	0.18	0
A002	21.1	0.98	0.24	1
A003	25.0	1.10	0.16	0
A004	22.3	1.05	0.20	1
...	...	...	...	...

Python 코드

```
# 모델 입력 변수 X (ID와 Target은 제외)
X = train.drop(columns=["id", "target"])
```

질문

id = 1, 2, 3, 4, ... 가
고장 여부 예측에 정말 의미 있는 정보일까요?

핵심 메시지

식별자와 **예축 변수**를 구분해야 합니다.

Classification인가? Regression인가?

1 Classification (분류 문제)

정해진 범주 중 하나를 예측하는 문제입니다.

예시

- **정상 / 고장** (이진 분류)
- **카테고리 분류** (다중 분류)
- **암 여부** (정상 / 암)
- **고객 이탈 여부** (이탈 / 유지)

데이터 예시

id	temperature	pressure	vibration	target
A001	23.5	1.02	0.18	0
A002	21.1	0.98	0.24	1
A003	25.0	1.10	0.16	0
A004	22.3	1.05	0.20	1
...	...	...	...	...

Target 값 형태

- 0 / 1
- 정상 / 고장
- A / B / C
- True / False
- Class 0, 1, 2 ...

Python 코드

```
# Target 변수 y 추출 (분류 문제)
y = train["target"]
```

2 Regression (회귀 문제)

연속된 숫자 값을 예측하는 문제입니다.

예시

- **가격 예측** (연속값)
- **전력 소비량** (연속값)
- **온도 예측** (연속값)
- **수요량 예측** (연속값)

데이터 예시

id	temperature	pressure	vibration	target
B001	24.1	1.01	0.19	103.2
B002	20.8	0.97	0.22	98.4
B003	23.6	1.08	0.17	121.8
B004	21.9	1.00	0.25	110.6
...	...	...	...	...

Target 값 형태

- 연속된 숫자 값
- 예: 103.2, 98.4
- 소수값 가능
- 범위가 넓을 수 있음

Python 코드

```
# Target 변수 y 추출 (회귀 문제)
y = train["target"]
```

주의할 점

Target이 숫자라고 해서
무조건 회귀 문제가 아닙니다.

예시 (분류 문제)

target	의미
0	정상
1	주의
2	위험

숫자이지만
정해진 범주이므로
Classification
(다중 분류)입니다.

핵심 메시지

**"데이터 타입이 아니라
값의 의미로 판단해야 합니다."**

평가 지표를 반드시 먼저 봐야 하는 이유

분류 문제 Classification

정해진 범주 중 하나를 예측하는 문제

- **Accuracy**: 전체 중 맞춘 비율
- **Precision**: 예측한 양성 중 실제 양성 비율
- **Recall**: 실제 양성 중 맞춘 비율
- **F1 Score**: Precision과 Recall의 조화평균
- **ROC-AUC**: 모든 임계값에서의 구분 능력

회귀 문제 Regression

연속된 숫자 값을 예측하는 문제

- **MAE**: 평균 절대 오차 (작을수록 좋음)
- **RMSE**: 평균 제곱근 오차 (큰 오차에 더 민감)
- **R^2**: 설명 가능한 분산 비율 (1에 가까울수록 좋음)

Accuracy의 함정

평가지표에 따라 좋은 예측의 정의가 다릅니다.

데이터 분포 (예시)

전체 1,000개

- 정상 (음성): 870개 (87%)
- 고장 (양성): 130개 (13%)

모든 데이터를 정상이라고 예측한 경우

실제 결과 \ 예측 결과	정상 예측 (0)	고장 예측 (1)
정상 (0)	870	0
고장 (1)	130	0

Accuracy 계산

$$\text{Accuracy} = \frac{870}{1000} = 0.87$$

모든 데이터를 정상이라고 예측해도 Accuracy가 0.87이 됩니다.

Accuracy 87%면 좋은 모델일까요?

반드시 그렇지 않습니다.

데이터가 불균형한 경우, 단순한 예측만으로도 높은 Accuracy가 나올 수 있습니다.

핵심 메시지 평가 지표에 따라 좋은 예측의 정의가 다릅니다. → 문제를 받으면 가장 먼저 **평가 지표를 확인해야 합니다.**

그래서 Baseline이 필요하다

Baseline이란?

"내 모델이 최소한 넘어야 할 기준점" 입니다.

좋은 모델인지 판단하기 위한 비교 기준이 되는 모델입니다.

중요한 포인트

- Dummy는 쓸모없는 모델이 아닙니다.
- 좋은 예측을 하기 위한 것이 아니라,
- **내 모델의 성능이 실제로 의미있는 개선인지 판단하기 위한 기준입니다.**

1 Baseline 모델의 단계

Level 0
Dummy Model

DummyClassifier
- 가장 단순한 기준선
- 데이터의 분포만 사용
- 예측 정보 없음

Level 1
Simple Linear Model

LogisticRegression
- 간단한 선형 모델
- 해석이 쉽고 빠르게 학습
- 기본적인 예측 성능 제공

Level 2
Tree-based Model

RandomForestClassifier
- 비선형 관계 학습
- 다양한 패턴 학습 가능
- Baseline 대비 성능 향상 기대

2 모델 비교 예시 (F1 Score)

Model	평균 F1	개선 폭	특징
Dummy (most_frequent)	0.7088	-	가장 단순한 기준선
Logistic Regression	0.7726	+0.0638	선형 모델 (해석 용이)
Random Forest	0.7409	+0.0321	비선형 모델 (복잡하지만 항상 더 좋지는 않음)

- Dummy 대비 성능이 얼마나 개선되었는지 확인해야 합니다.
- 모델이 복잡하다고 해서 항상 더 좋은 성능을 내는 것은 아닙니다.
- 같은 데이터, 같은 검증 조건에서 비교해야 합니다.

3 Dummy Model 예제

Python 코드 (scikit-learn)

```python
from sklearn.dummy import DummyClassifier

model = DummyClassifier(
    strategy="most_frequent")
)
```

결과 해석 예시

Dummy Model **F1 = 0.71** → 내 모델 **F1 = 0.72**

겉보기에는 0.72로 더 높지만,
실제 개선 폭은 **+0.01**에 불과합니다.

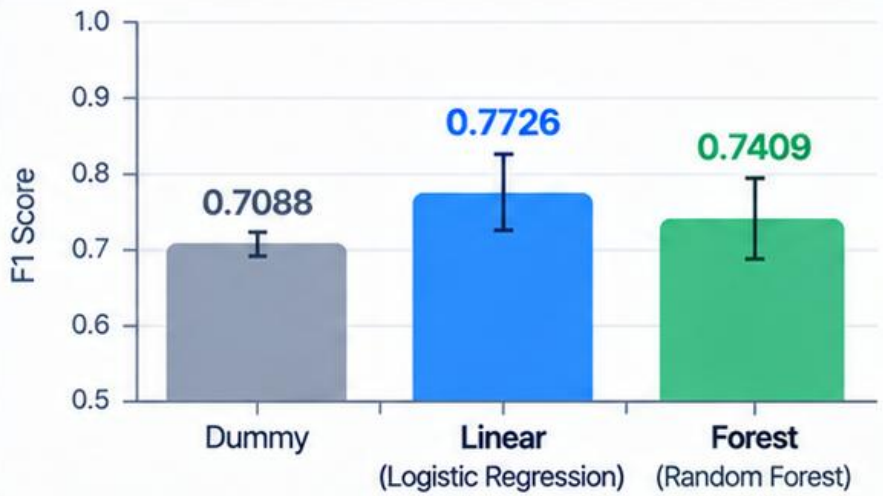
→ 이 정도 차이가 실제로 의미 있는 개선인지 통계적으로 확인해야 합니다.

실제 데이터에서의 Baseline 비교 예시

모델별 성능 비교 결과 (F1 Score)

Model	Mean F1	SD	SE	95% Confidence Interval
Dummy (most_frequent)	**0.7088**	0.0018	0.0006	[0.7076, 0.7101]
1 **Linear** (Logistic Regression)	**0.7726**	0.0316	0.0100	**[0.7500, 0.7952]**
3 **Forest** (Random Forest)	**0.7409**	0.0316	0.0100	[0.7183, 0.7635]

모델별 F1 Score (평균과 95% 신뢰구간)



핵심 메시지

"모델의 복잡성과 성능은 같은 것이 아니다."

- 많은 사람들이 Random Forest가 더 좋은 성능을 낼 것이라고 생각하지만,
- 실제 결과는 Logistic Regression이 가장 높은 F1을 보였습니다.
- 따라서 모델의 복잡성만으로 성능을 판단할 수 없습니다.

반드시 같은 조건에서 비교해야 한다

- **같은 데이터 사용**
  동일한 전처리, 동일한 데이터셋
- **같은 Fold (데이터 분할)**
  동일한 교차검증 폴드
- **같은 Metric**
  예: F1 Score
- **같은 Evaluation Procedure**
  동일한 학습/검증 절차, 하이퍼파라미터 설정

잘못된 비교 예시

Logistic Regression	Random Forest
Seed = 42	Seed = 123
Validation = 20%	Validation = 30%

데이터 분할 조건이 달라
점수가 달라도 어떤 모델이 더 좋은지
판단할 수 없습니다.

올바른 비교 방법

- 같은 데이터 사용
- 같은 Fold (교차검증)
- 같은 Metric
- 같은 Evaluation Procedure

이 조건에서 비교해야
모델의 성능을 공정하게
평가할 수 있습니다.

문제 정의와 Baseline 만들기

실습 1 — 데이터를 처음 받았을 때

1 데이터 크기 확인하기

Train과 Test의 행 수와 열 수를 확인합니다.

```
train.shape
test.shape
```

```
train.shape
(1000, 11)

test.shape
(500, 10)
```

? 질문

Train과 Test의 행 수는 각각 몇 개인가?

2 컬럼 확인하기

데이터의 앞부분을 보고 어떤 컬럼이 있는지 확인합니다.

```
train.head()
```

	id	temperature	pressure	vibration	target
0	A001	23.5	1.02	0.18	0
1	A002	21.1	0.98	0.24	1
2	A003	25.0	1.10	0.16	0
3	A004	22.3	1.05	0.20	1
4	A005	24.1	1.08	0.19	0

? 질문

어떤 column들이 존재하는가?

3 데이터 정보 확인하기

데이터 타입, 결측, 행/열 수를 확인합니다.

```
train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 11 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   id           1000 non-null   object
 1   temperature  1000 non-null   float64
 2   pressure     1000 non-null   float64
 3   vibration    995 non-null    float64
 4   target       1000 non-null   int64
dtypes: float64(3), int64(1), object(7)
memory usage: 86.1+ KB
```

? 질문

- Data Type은 무엇인가?
- Missing은 있는가?
- 행 수와 열 수는 각각 몇 개인가?

4 결측값 확인하기

어떤 컬럼에 missing이 있는지 확인합니다.

```
train.isnull().sum()
```

```
id             0
temperature    0
pressure       0
vibration      5
target         0
dtype: int64
```

? 질문

어떤 column에 missing이 있는가?

5 Target 분포 확인하기

클래스 비율이 균형적인지 확인합니다.

```
train[target].value_counts()
```

```
target
0    870
1    130
Name: count, dtype: int64
```

```
train[target].value_counts
(normalize=True)
```

```
target
0    0.87
1    0.13
Name: proportion, dtype: float64
```

? 질문 클래스가 균형적인가?

Data Contract — 데이터가 우리가 기대한 구조를 갖고 있는지 검사한다

주요 확인 항목

- Train에 Target이 있는가?
- Test에는 Target이 없는가?
- ID가 중복되지 않는가?
- 필수 Column이 존재하는가?
- Sample Submission과 Test ID가 대응되는가?
- Target에 결측값이 없는가?

Data Contract 실행 예시

```
data_contract(train, test, sample_submission
, id_col="id", target_col="target")
```

```
[INFO] train shape: (1000, 11)
[INFO] test shape: (500, 10)
[PASS] Train에 target 컬럼이 있습니다.
[PASS] Test에 target 컬럼이 없습니다.
[PASS] ID 중복이 없습니다.
[PASS] 필수 컬럼이 모두 존재합니다.
[PASS] Test ID와 submission ID가 일치합니다.
[PASS] Target에 결측값이 없습니다.
```

Fail 예시 (문제가 있는 경우)

```
data_contract(train, test, sample_submission
, id_col="id", target_col="target")
```

```
[INFO] train shape: (1000, 11)
[INFO] test shape: (500, 10)
[FAIL] Test에 target 컬럼이 존재합니다.
[FAIL] ID가 중복됩니다. (중복 개수: 3)
[PASS] 필수 컬럼이 모두 존재합니다.
[PASS] Test ID와 submission ID가 일치합니다.
[FAIL] Target에 결측값이 있습니다. (개수: 5)
```

1차시
10

문제 정의와 Baseline 만들기

Test와 Sample Submission의 ID 순서가 다를 수 있다

제출 파일은 ID 기준으로 예측값이 매칭되어야 합니다. 단순히 순서대로 넣으면 잘못된 사람에게 예측값이 들어갈 수 있습니다.

Test 데이터 (예시)

행 번호	id	feature1	feature2	...
0	A01	1.2	0.5	...
1	A02	0.8	1.1	...
2	A03	2.1	0.3	...

Sample Submission (예시)

행 번호	id	target
0	A03	0
1	A01	0
2	A02	0

모델 예측 결과 (Test 순서로 파생된 예측값)

행 번호	id (Test 순서)	pred (예측값)
0	A01	0.1
1	A02	0.7
2	A03	0.3

잘못된 방법 (순서대로 넣기)

Test와 Sample의 ID 순서가 다를 때,
예측값을 그대로 넣으면 다른 행에 예측값이 들어갑니다.

잘못된 코드 예시

```
# 예측값을 고냥 순서대로 넣음
submission = sample.copy()
submission["target"] = pred  # ❌
```

제출 파일 결과 (잘못 매칭됨)

행 번호	id	target (잘못된 예측값)		
0	A03	0.1	❌	(A01의 예측이 들어감)
1	A01	0.7	❌	(A02의 예측이 들어감)
2	A02	0.3	❌	(A03의 예측이 들어감)

올바른 방법 (ID 기준으로 매칭)

ID를 기준으로 예측값을 연결하면
어떤 순서라도 올바르게 매칭됩니다.

올바른 코드 예시

```
# ID를 기준으로 매칭
submission = sample.copy()
submission = submission.merge(
    pd.DataFrame({
      "id": test["id"],
      "target": pred
    }),
    on="id", how="left"
)
```

제출 파일 결과 (올바르게 매칭됨)

행 번호	id	target (정확한 예측값)
0	A03	0.3
1	A01	0.1
2	A02	0.7

핵심 메시지

- Test와 Sample Submission의 ID 순서는 다를 수 있습니다.
- 제출 파일은 반드시 ID를 기준으로 예측값을 연결해야 합니다.
- 단순히 순서대로 넣으면 잘못된 사람에게 예측값이 들어갈 수 있습니다.

"제출은 항상 ID 기준으로 매칭한다."

1차시 11

문제 정의와 Baseline 만들기

Baseline 실습부터 첫 Submission까지

1 Dummy 모델

가장 단순한 기준선 모델

Python 코드

```python
from sklearn.dummy import
 DummyClassifier

dummy = DummyClassifier(
    strategy="most_frequent"
)
```

설정 의미

- 가장 많이 등장하는 클래스로 예측
- 좋은 예측을 위한 모델이 아니라, 비교 기준점으로 사용

2 Logistic Regression

간단한 선형 모델

Python 코드

```python
from sklearn.linear_model import
 LogisticRegression

lr = LogisticRegression()
```

설정 의미

- 빠르고 간단한 학습
- 해석이 쉽고 안정적인 성능
- 강력한 Baseline 모델

3 Random Forest

비선형 모델 (좀 더 복잡한 모델)

Python 코드

```python
from sklearn.ensemble import
 RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=200,
    random_state=42
)
```

설정 의미

- 다양한 패턴을 학습할 수 있는 모델
- 파라미터가 많고 복잡함
- 항상 더 좋은 성능을 보장하지 않음

4 모델 비교 결과 (예시)

교차검증 결과 (F1 Score)

Model	Mean F1	SD	SE	95% CI
Dummy	0.7088	0.0018	0.0006	[0.7076, 0.7101]
Logistic Regression	0.7726	0.0316	0.0100	[0.7500, 0.7952]
Random Forest	0.7409	0.0316	0.0100	[0.7183, 0.7635]

? 학습 후 질문

Q1 가장 성능이 좋은 모델은?
A: Logistic Regression

Q2 가장 복잡한 모델은?
A: Random Forest

Q3 복잡한 모델이 항상 성능이 좋은가?
A: 아니다 (항상 더 좋지 않다)

5 전체 데이터로 다시 학습

가장 좋은 모델(Logistic Regression) 선택 후 전체 Train 데이터로 다시 학습

Python 코드

```python
model = LogisticRegression()
model.fit(X_train_all, y_train_all)
```

6 Test 예측

학습한 모델로 Test 데이터 예측

Python 코드

```python
pred = model.predict(X_test)
```

7 Submission 파일 생성

Sample Submission 형식에 맞춰 작성

Python 코드

```python
submission = sample_submission.copy()
submission["target"] = pred
submission.to_csv("submission.csv",
                  index=False)
```

8 저장했다고 끝이 아니다!

파일을 다시 읽어서 형식과 내용을 확인

Python 코드

```python
check = pd.read_csv("submission.csv")
check.shape
check.head()
check.isnull().sum()
```

핵심 메시지

- 여러 모델을 같은 조건에서 비교해야 합니다. (동일한 데이터, 동일한 Fold, 동일한 Metric)
- 모델의 복잡성과 성능은 같은 것이 아닙니다.
- 제출 파일은 반드시 다시 읽어서 형식과 내용을 확인해야 합니다.

2차시
01

데이터를 읽고 문제를 발견하는 힘

통계로 데이터를 진단하는 방법

EDA를 그림 그리기로 끝내지 않고, 데이터에서 문제를 찾아 전처리와 모델링의 근거를 만드는 시간입니다.

차시 개요

주제

EDA를 그림 그리기로 끝내지 않기

단순히 그래프를 그리는 것이 아니라, 데이터를 통계적으로 진단하고 **전처리 결정의 근거**를 만드는 방법을 학습합니다.

핵심 질문

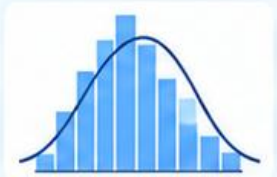
1. 이 변수는 왜 이렇게 생겼는가?
2. 이 값은 정말 이상치인가?
3. 결측치는 왜 발생했는가?
4. Target과 실제로 관계가 있는가?
5. Train과 Test는 같은 데이터라고 볼 수 있는가?

데이터 진단의 전체 흐름 (6단계)

분포 → 이상치 → 결측치 → 변수-Target 관계 → 공선성 → Train-Test 분포 차이

1 분포 진단

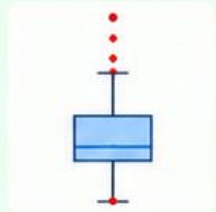
분포가 치우쳤는가?



- Mean / Median
- IQR / MAD
- Skewness
- Kurtosis
- 정규성 검정

2 이상치 진단

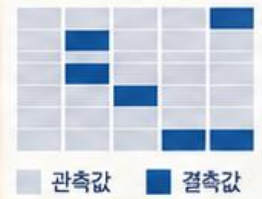
정말 오류인가?



- IQR Rule
- Robust Z-score
- 삭제 여부는 별도 판단

3 결측치 진단

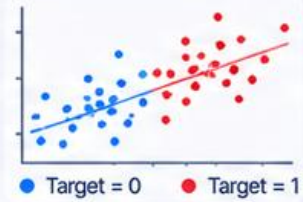
왜 발생했는가?



- MCAR / MAR / MNAR
- 결측 패턴 분석
- Missing Indicator
- 결측과 Target 관계

4 변수-Target 관계

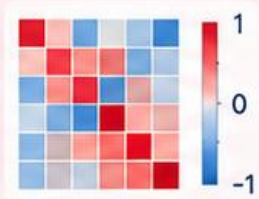
실제로 관계가 있는가?



- Pearson / Spearman
- t-test / Mann-Whitney
- Chi-square test
- Effect size
- p-value 해석

5 다중공선성

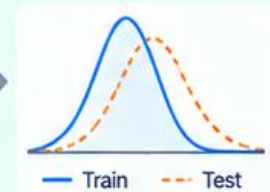
같은 정보를 반복하고 있는가?



- 상관계수 행렬
- VIF (Variance Inflation Factor)
- 자동 제거가 아님

6 Train-Test 분포 차이

같은 데이터라고 볼 수 있는가?



- KS Test
- PSI (Population Stability Index)
- Train/Test Classifier (Adversarial Validation)

학습목표 — 2차시가 끝나면 다음을 할 수 있습니다.

1. 평균, 표준편차뿐 아니라 Median, IQR, MAD, Skewness, Kurtosis를 함께 해석할 수 있다.
2. 정규성 검정의 p-value만 보고 변환을 결정하지 않는다.
3. 이상치를 통계적으로 탐지하고, 삭제 여부는 별도로 판단할 수 있다.
4. MCAR / MAR / MNAR의 차이를 사례 수준으로 설명할 수 있다.
5. p-value와 effect size를 구분할 수 있다.
6. 수치형/범주형 변수에 적절한 연관성 지표를 선택할 수 있다.
7. VIF가 높다는 이유만으로 변수를 자동 제거하지 않는다.
8. KS, PSI, adversarial validation으로 Train/Test shift를 진단하고, 진단 결과를 실제 전처리 결정으로 연결할 수 있다.

2차시
03

통계로 데이터를 진단하는 방법

기술통계 — describe()를 제대로 읽기

df.describe()에서 mean과 std만 보는 것이 아니라, 6가지 핵심 지표를 함께 확인합니다.

핵심 메시지

"평균과 표준편차만으로는
데이터의 모습을 충분히 이해할 수 없다."

1 df.describe() 예시

	count	mean	std	min	25%	50%	75%	max
temperature	1000	23.5	5.2	5.1	19.8	22.9	26.7	52.1
pressure	1000	180.4	60.1	10.2	140.3	175.1	210.6	520.3
vibration	1000	0.25	0.18	0.01	0.12	0.20	0.34	1.50

2 함께 확인해야 할 6가지 지표

Mean (평균)
데이터의 중심 위치
(모든 값의 평균)

Median (중앙값)
데이터의 중앙 위치
(50% 지점)

IQR (사분위범위)
Q3 − Q1
중앙 50% 범위

MAD (중앙절대편차)
median(|x − median|)
이상치에 강한 퍼짐 지표

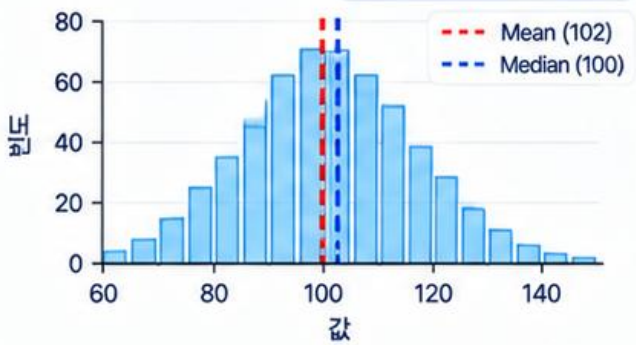
Skewness (왜도)
분포의 치우침 정도
(0: 대칭, >0: 오른쪽 꼬리)

Kurtosis (첨도)
분포의 뾰족함과 꼬리 두께
(0: 정규분포와 유사)

3 Mean과 Median 비교

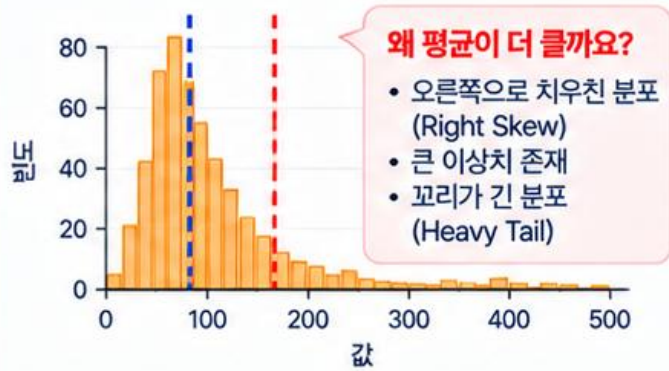
Feature A
(거의 대칭인 분포)

Mean = 102
Median = 100
큰 차이가 없음



Feature B
(오른쪽으로 치우친 분포)

Mean = 180
Median = 95)



왜 평균이 더 클까요?

- 오른쪽으로 치우친 분포 (Right Skew)
- 큰 이상치 존재
- 꼬리가 긴 분포 (Heavy Tail)

4 IQR (Interquartile Range)



- 중앙 50% 데이터의 범위를 나타냅니다.
- 극단값의 영향을 평균이나 표준편차보다 적게 받습니다.

5 MAD (Median Absolute Deviation)

$$MAD = \text{median}(|x_i - \text{median}(x)|)$$

중앙값으로부터의 절대편차의 중앙값

표준편차 (Std)	MAD
• 평균 기준으로 퍼짐 측정 • 이상치의 영향을 크게 받음	• 중앙값 기준으로 퍼짐 측정 • 이상치의 영향을 작게 받음

2차시
04

통계로 데이터를 진단하는 방법

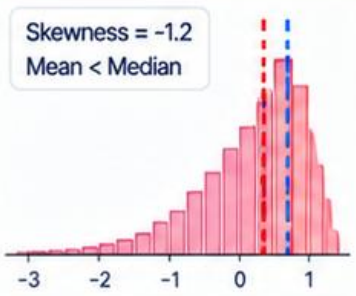
Skewness와 Kurtosis — 분포의 모양을 읽기

Skewness는 분포의 치우침 정도, Kurtosis는 꼬리와 극단값 성향을 나타내는 지표입니다.

1 Skewness (왜도) — 분포가 어느 방향으로 치우쳤는가?

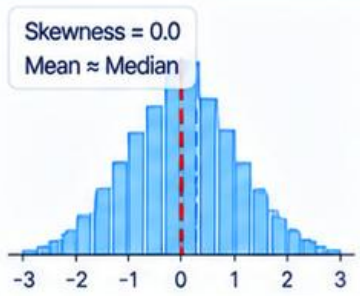
Skewness는 분포의 비대칭 정도를 나타내는 지표로, 평균과 중앙값의 위치 차이와 꼬리의 방향을 확인할 수 있습니다.

Skewness < 0 (왼쪽 꼬리)



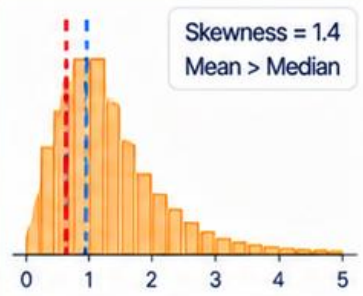
- 왼쪽 꼬리가 긴 분포
- 평균이 중앙값보다 작음

Skewness ≈ 0 (대칭)



- 좌우가 대칭인 분포
- 평균과 중앙값이 거의 같음

Skewness > 0 (오른쪽 꼬리)



- 오른쪽 꼬리가 긴 분포
- 평균이 중앙값보다 큼

? 질문 로그 변환(Log transformation)은 무엇을 고치기 위해 주로 사용하는가?

A 치우침(Skewness)을 줄이기 위해 주로 사용합니다.

로그 변환 전 (오른쪽으로 치우진 분포)



Log 변환

$x' = \log(1 + x)$

로그 변환 후 (더 대칭적인 분포)



2 Kurtosis (첨도) — 꼬리와 극단값 성향은 어떤가?

Kurtosis는 분포의 뾰족함과 꼬리의 두꺼움을 나타내는 지표로, 극단값(이상치) 성향을 함께 확인할 수 있습니다.
(보통 정규분포의 kurtosis = 3, excess kurtosis = 0으로 해석합니다.)

Excess Kurtosis < 0 (평평한 분포)



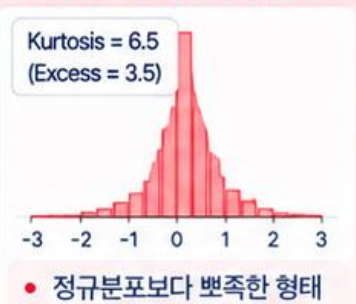
- 정규분포보다 평평한 형태
- 꼬리가 얇음 (극단값 적음)

Excess Kurtosis ≈ 0 (정규분포와 유사)



- 정규분포보 유사한 형태
- 꼬리와 뾰족함이 적절함

Excess Kurtosis > 0 (뾰족하고 꼬리가 두꺼운 분포)



- 정규분포보다 뾰족한 형태
- 꼬리가 두꺼움 (극단값 많음)

실제 예시 — skewness는 0인데, Kurtosis가 음수인 경우

다음과 같이 skewness가 거의 0이지만,
excess kurtosis가 -1.2인 경우를 생각해봅니다.

- Skewness ≈ 0 → 좌우 대칭
- Excess kurtosis ≈ -1.2 → 균등분포에 가까운 평평한 형태
- Shapiro 정규성 검정은 기각될 수 있음
- (치우침 때문이 아니라, 정규분포보다 평평하기 때문에)

예시 분포



핵심 메시지

- Skewness는 분포의 치우침 정도를 나타내며, 로그 변환은 치우침을 줄이기 위해 사용한다.
- Kurtosis는 단순히 뾰족함만이 아니라 꼬리와 극단값 성향을 함께 보는 지표이다.

2차시
05

통계로 데이터를 진단하는 방법

정규성 검정의 함정 — p-value만 보고 변환하면 안 된다

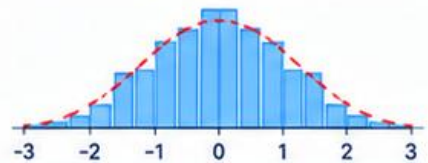
정규성 검정은 '정규분포와 얼마나 다른가'를 매우 엄격하게 판단합니다. 따라서 p-value만 보고 로그 변환을 결정하면 잘못된 전처리가 될 수 있습니다.

1 실제 사례

Feature	Skewness	Kurtosis (Excess)	Shapiro p-value
vibration	-0.03	-1.21	7.4e-16
hours	0.02	-1.18	3.1e-15
load	0.01	-1.23	5.2e-14

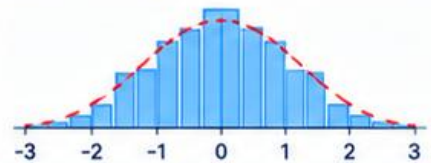
2 예시: 정규성 검정은 기각되지만, 분포는 정규분포와 유사

vibration
Skew = -0.03, Kurtosis = -1.21
Shapiro p = 7.4e-16



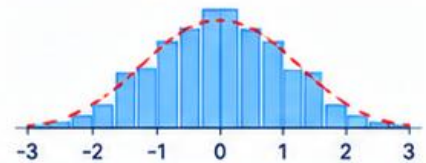
Skewness는 거의 0이지만,
kurtosis가 -1.2로 평평한 분포
→ Shapiro 검정 기각

hours
Skew = 0.02, Kurtosis = -1.18
Shapiro p = 3.1e-15



Skewness는 거의 0이지만,
kurtosis가 -1.2로 평평한 분포
→ Shapiro 검정 기각

load
Skew = 0.01, Kurtosis = -1.23
Shapiro p = 5.2e-14



Skewness는 거의 0이지만,
kurtosis가 -1.2로 평평한 분포
→ Shapiro 검정 기각

3 많은 학생들이 하는 잘못된 판단

Shapiro p = 7e-16
(p < 0.05) 이니까
로그 변환해야 하나요?



많은 학생들의 답변
"네!"

4 하지만 정답은?

Skewness ≈ 0이라면
로그 변환을 할 이유가 거의 없습니다.

- 정규성 검정은 표본 수가 크면
  아주 작은 차이도 기각할 수 있습니다.
- 이 예시는 치우침이 아니라
  kurtosis가 -1.2로 평평한 분포(균등분포에 가까움)
  이기 때문에 기각된 사례입니다.

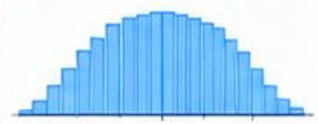
5 정규성 검정이 기각되는 이유는?

꼬리가 두꺼운 분포
(kurtosis > 0)



뾰족하고 극단값이 많은 분포
→ **정규성 검정 기각**

꼬리가 얇은(평평한) 분포
(kurtosis < 0)



균등분포에 가까운 평평한 분포
→ **정규성 검정 기각**

6 꼭 기억할 오개념

오개념 1
"정규성 검정이 기각되면
feature를 정규분포로
만들어야 한다."
→ 아니다.

오개념 2
"선형모델은 모든 feature가
정규분포여야 한다."
→ 아니다.

핵심 메시지

- 정규성 검정의 p-value만 보고 로그 변환을 결정하지 않는다.
- 정규성 검정이 기각되더라도 치우침이 없는 경우(예: kurtosis -1.2로 평평한 분포) 로그 변환이 필요하지 않을 수 있다.
- 선형모델의 정규성 가정은 주로 잔차의 정규성과 관련되며, feature 자체의 정규성이 필수 조건은 아니다. 트리 기반 모델은 단조변환의 영향도 비교적 적다.

2차시
06

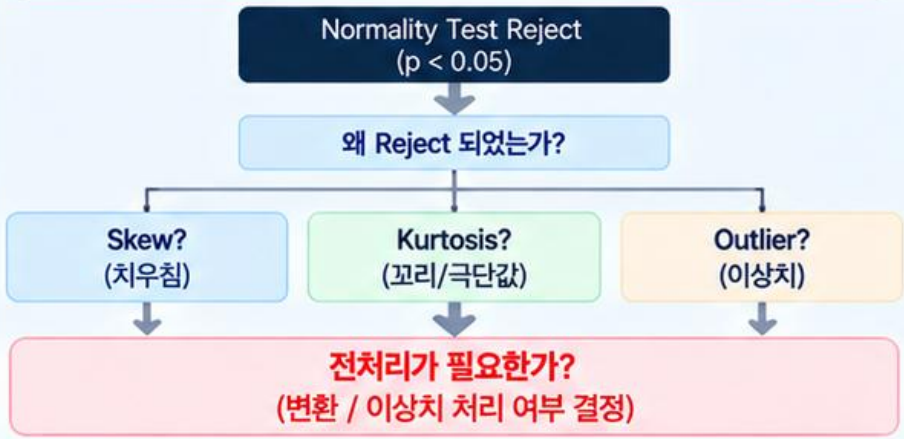
통계로 데이터를 진단하는 방법

정규성 검정 이후의 판단과 이상치 진단

정규성 검정이 기각되었다고 해서 자동으로 변환하지 않고, 이상치는 삭제 대상이 아니라 '판단이 필요한 후보'입니다.

1 첫 번째 핵심 메시지

"통계적으로 정규분포가 아니라고 해서 자동으로 변환하지 않는다."

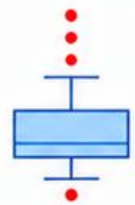


2 이상치 진단 방법

A IQR Rule (사분위수 범위 규칙)

$$Lower = Q_1 - 1.5 \times IQR$$
$$Upper = Q_3 + 1.5 \times IQR$$

$$IQR = Q_3 - Q_1$$



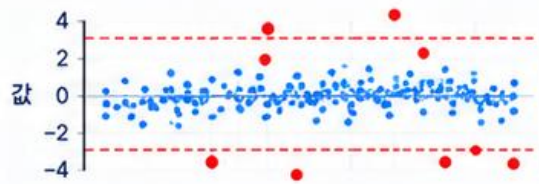
- Q3 + 1.5×IQR 초과 → 이상치 후보
- Q1 - 1.5×IQR 미만 → 이상치 후보

- 중앙 50% 데이터 범위를 기준으로 탐지
- 극단값의 영향을 평균/표준편차보다 적게 받음
- 분포의 형태와 크기에 관계없이 적용 가능

B Robust Z-score (강건한 Z-score)

$$z_r = 0.6745 \frac{x - Median(x)}{MAD}$$

일반적인 기준
$|z_r| > 3.5$
→ 이상치 후보



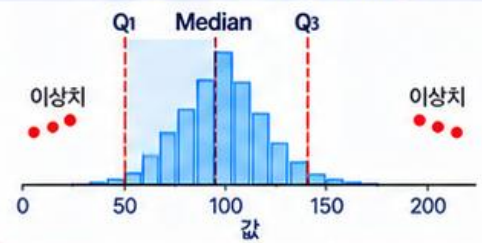
- Median과 MAD를 사용한 강건한 방법
- 평균과 표준편차가 이상치의 영향을 받는 문제를 해결
- 비대칭 분포에서도 안정적으로 작동

3 이상치 판단 – 무조건 삭제하지 않는다

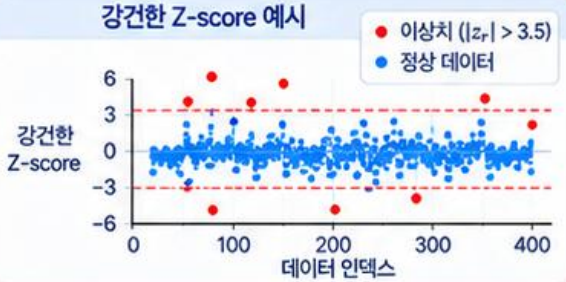
상황	예시	판단 및 대응
물리적으로 불가능한 값	온도 -100℃, 나이 300세	오류 가능성 → 수정/제거 검토
드물지만 가능한 값	매우 큰 지출 금액, 특정 장비의 최대 압력	유지 (실제 데이터일 수 있음)
모델이 지나치게 민감한 경우	일부 이상치로 인해 모델 성능이 불안정	Clipping / Robust 모델 (예: 트리 모델, Huber Loss)
단순히 prediction error가 큰 값	예측이 어려운 구간의 정상 데이터	삭제하면 안 됨 (모델이 아직 설명 못한 사례)

4 이상치 진단 예시

정상 분포와 이상치 (IQR Rule)



강건한 Z-score 예시



핵심 메시지

- 이상치는 통계적 후보일 뿐, 삭제 명령이 아니다. 반드시 데이터의 의미와 도메인 지식을 함께 고려하여 판단한다.
- 정규성 검정의 p-value는 '정규분포와의 차이'를 의미할 뿐, 변환이 반드시 필요하다는 뜻은 아니다.

2차시
08

통계로 데이터를 진단하는 방법

결측치(Missing Value) — 중요한 것은 몇 %가 아니라 왜 비었는가?

결측치는 단순히 값을 채우는 문제가 아니라, 왜 비었는지를 이해해야 올바른 전처리 방법을 선택할 수 있습니다.

1 결측치의 세 가지 유형

MCAR
Missing Completely At Random

결측 여부가
관측 변수와도 무관
미관측 값과도 무관

예시



통신이
무작위로 끊김

- 완전히 무작위로 발생하는 결측
- 관측된 데이터로는 설명이 어려움

MAR
Missing At Random

결측이 **다른 관측 변수로**
설명됩니다.

예시



특정 zone에서만
센서 통신이 불안정

- 다른 관측 변수(에: zone, 시간 등)와 관련
- 관측된 정보를 이용해 결측을 설명할 수 있음

MNAR
Missing Not At Random

결측이 **가려진 값 자체와**
관련됩니다.

예시



온도가 너무 높으면
센서가 고장나
측정되지 않음

- 실제 값 자체와 관련된 결측
- 관측된 정보만으로 설명하기 어려움

2 MNAR이 왜 위험한가?

**값이 클수록 결측이 발생하는 경우, 단순한 대체는
데이터의 의미를 왜곡하여 잘못된 결론을 만들 수 있습니다.**

실제 상황
온도가 매우 높은 상태



Temperature
= 100℃

→

센서 고장으로 결측 발생



관측값
= NaN

→

단순 대체 (예: Median)



NaN → 23℃
(전체 데이터의 중앙값)

**위험한 고온 상황을 관병흰 온도로 바꾸는 오류가 발생할 수 있습니다.
이는 모델 학습에 체계적 편향을 유발할 수 있습니다.**

3 결측 패턴 확인 실습

```
# 각 결럼의 결측 비율 확인
train.isnull().mean().sort_values(ascending=False)
```

feature	missing_rate	missing_count
vibration	0.049	49
temperature	0.035	35
pressure	0.012	12
load	0.005	5
hours	0.002	2

4 Missing Indicator 생성

```
# 결측 여부를 나타내는 변수 생성
train["temperature_missing"] = (
    train["temperature"].isna().astype(int)
)
```

temperature	temperature_missing
23.5	0
NaN	1
18.2	0
25.1	0
NaN	1

5 결측과 Target의 관계 확인

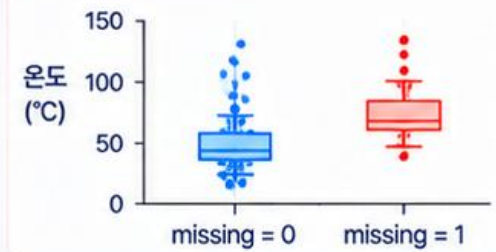
수치형 변수 (회귀/분류 공통)

- 결측 고룹 vs 비결측 그룹 비교
- t-test / Mann-Whitney U
- 효과크기 (Cohen's d)

범주형 변수 (분류)

- Missing indicator vs Target
- Chi-square test
- Cramér's V

질문 temperature_missing
자체가 target과 관련이 있는가?



핵심 메시지

- 결측치는 몇 %인지보다 왜 비었는지가 더 중요합니다.
- 특히 MNAR은 단순한 대체로 해결할 수 없으며, 도메인 이해와 추가적인 분석이 필요합니다.

2차시 20

통계로 데이터를 진단하는 방법

결측과 Target 관계 분석 — 어떤 검정을 사용할까?

결측 여부 자체가 Target과 관련이 있는지 통계적으로 검정하여, 결측의 원인을 추정하고 적절한 처리 방법을 결정합니다.

1 Classification (범주형 Target)

Missing indicator와 Target의 독립성 검정

	Target = 0	Target = 1	Total
Missing = 0 (관측값)	850	100	950
Missing = 1 (결측값)	30	20	50
Total	880	120	1,000

사용하는 통계 검정

- **Chi-square test** (카이제곱 검정)
  - 결측 여부와 Target이 독립적인지 검정
- **Cramér's V**
  - 연관성의 크기(효과크기) 측정 (0 ~ 1, 클수록 강한 연관)

Target 비율 비교 예시

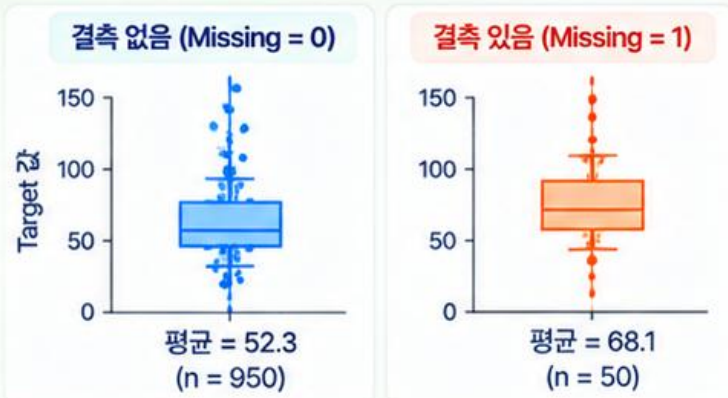


분석 결과 예시

Chi-square p-value	0.003
Cramér's V	0.18
해석	결측 여부와 Target에 통계적으로 유의한 연관이 있음 (약한 연관)

2 Regression (연속형 Target)

결측 그룹과 비결측 그룹의 Target 분포 차이 검정



사용하는 통계 검정

- **t-test (Welch t-test)**
  - 두 그룹의 평균 차이 검정 (정규성 가정이 어려운 경우)
- **Mann–Whitney U test**
  - 두 그룹의 중앙값 차이 검정 (비모수 검정)
- **Cohen's d**
  - 효과 크기 측정 (0.2: 작은 효과, 0.5: 중간, 0.8: 큰 효과)

분석 결과 예시

t-test p-value	0.008
Cohen's d	0.42
해석	결측 그룹의 Target 값이 통계적으로 유의하게 높음 (중간 정도의 효과 크기)

3 통계 검정 결과 해석 시 주의사항

잘못된 표현

"p > 0.05이므로 MCAR이다."

- 유의하지 않다고 해서 '완전히 무관하다고 단정할 수 없음
- 검정력이 부족할 수 있음 (결측 건수가 적은 경우)

올바른 표현

"현재 데이터에서 MAR의 증거를 찾지 못했다."

- Not Significant ≠ No Effect
- 현재 데이터에서 통계적으로 유의한 연관이 발견되지 않았다는 의미
- 더 많은 데이터가 있다면 결과가 달라질 수 있음

4 왜 Not Significant ≠ No Effect 인가?

- 표본 수가 작으면 작은 효과를 발견하기 어려움 (검정력 부족)
- p-value는 우연에 의한 차이일 가능성을 평가할 뿐, 효과의 존재 여부를 증명하지는 않음
- 효과 크기(Cohen's d, Cramér's V)를 함께 보고 실질적인 차이가 있는지 판단해야 함

5 핵심 메시지

"p-value만 보지 말고, 효과 크기와 도메인 지식을 함께 고려하여 결측의 원인을 해석한다."

2차시
09

통계로 데이터를 진단하는 방법

Feature와 Target 관계 — 어떤 변수가 실제로 도움이 되는가?

변수의 타입과 Target의 형태에 따라 적절한 통계 분석 방법을 선택하여, 실제로 예측에 도움이 되는 변수를 판단합니다.

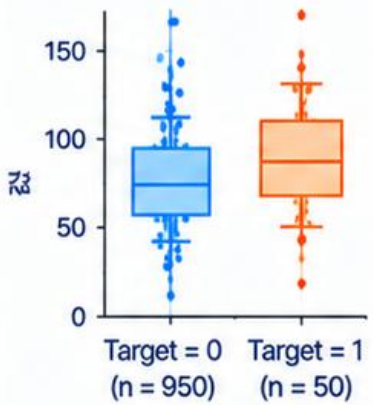
1 수치형 Feature ↔ 이진 Target

두 고룹(예: Target = 0, 1)의 분포 차이를 비교

사용하는 분석 방법

- **Welch t-test**
  (두 그룹의 평균 차이 검정, 분산이 다를 때)
- **Mann–Whitney U**
  (비모수 검정, 정규성 가정 불필요)
- **Cohen's d**
  (효과 크기 측정)
- **Univariate AUC**
  (단변량 예측력 평가, 분류 문제에서 자주 사용)

예시: feature의 분포 비교



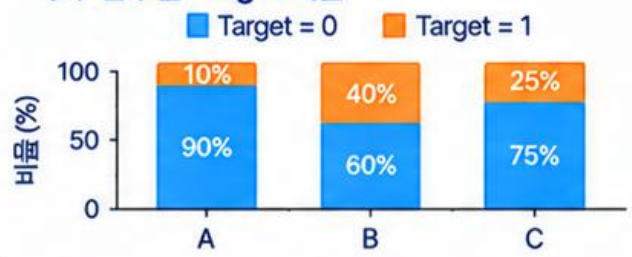
2 범주형 Feature ↔ Target

범주 간 Target 비율의 차이를 검정

사용하는 분석 방법

- **Chi-square Test**
  (범주형 변수의 독립성 검정)
- **Cramér's V**
  (연관성의 크기 측정, 0~1, 를수록 강한 연관)

예시: 범주별 Target 비율



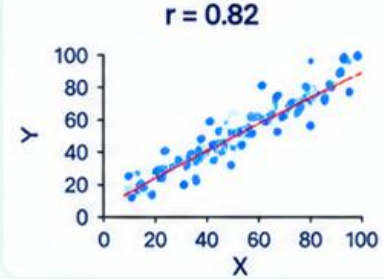
3 수치형 Feature ↔ 수치형 Target

두 변수의 상관관계를 분석

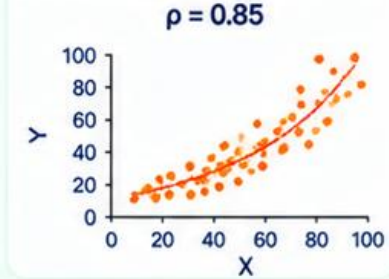
사용하는 분석 방법

- **Pearson Correlation**
  (선형 관계 측정)
- **Spearman Correlation**
  (순위 기반 상관관계, 단조 관계 측정)

예시: 선형 관계 (Pearson)



예시: 비선형 단조 관계 (Spearman)



4 Pearson과 Spearman 비교

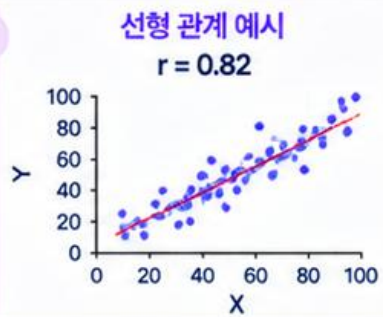
Pearson 상관계수 (r)

선형 관계 측정

$$r = \frac{Cov(X,Y)}{\sigma_X \sigma_Y}$$

- 두 변수의 선형적 관계의 강도와 방향을 측정
- 값의 크기(원분 값)를 그대로 사용
- 이상치에 민감

선형 관계 예시



Spearman 순위상관계수 (ρ)

단조 관계 측정

- 두 변수의 순위(rank) 간 상관관계 측정
- 직선 관계가 아니더라도 단조 증가/감소 관계라면 잘 잡아냄
- 이상치의 영향을 덜 받음

곡선형 단조 관계 예시



핵심 메시지

- 수치형-이진 Target은 평균 차이와 효과 크기를 함께 분다.
- 범주형 변수는 카이제곱 검정과 Cramér's V로 연관성을 평가한다.
- 수치형-수치형 Target은 Pearson과 Spearman을 모두 확인한다.
- 변수의 중요성은 p-value만으로 판단하지 않고, 효과 크기와 실제 예측 성능(AUC 등)을 함께 고려한다.

2차시
10

통계로 데이터를 진단하는 방법

t-test와 Cohen's d를 같이 보는 이유 — p-value와 Effect Size

p-value는 '차이가 있는가?'를, Effect Size는 '그 차이가 얼마나 큰가?'를 보여줍니다. 두 값을 함께 봐야 실제로 의미 있는 변수를 판단할 수 있습니다.

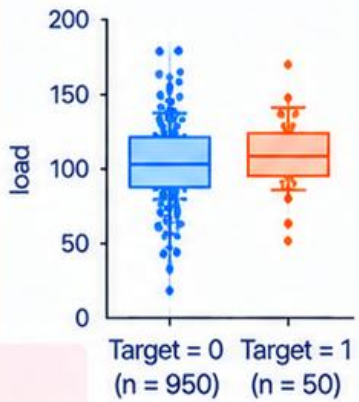
1 실제 예시: p-value는 매우 작지만 효과는 작다

Feature: load (Target = 0 vs 1)

Group	n	평균	표준편차
Target = 0	950	102.1	28.5
Target = 1	50	110.5	29.1

t-test p-value	Cohen's d	단변량 AUC
0.00004	0.26	0.576

load의 그룹별 분포



- p-value가 매우 작아 통계적으로 유의하지만,
- Cohen's d=0.26, AUC=0.576으로 효과 크기는 작습니다.
- 즉, 실질적인 예측력은 크지 않은 변수입니다.

2 p-value와 Effect Size의 의미

p-value

"차이가 있는가?"

관측된 차이가 우연에 의해 발생했을 가능성을 나타냄



p-value < 0.05

→ 두 그룹의 평균에 통계적으로 유의한 차이가 있음

Effect Size (Cohen's d)

"그 차이가 얼마나 큰가?"

두 그룹의 평균 차이가 얼마나 큰지를 나타냄 (단위에 독립적)

$$d = \frac{\bar{x}_1 - \bar{x}_0}{s_{pooled}}$$

Cohen's d	효과 크기 해석
0.2	작은 효과
0.5	중간 효과
0.8	큰 효과

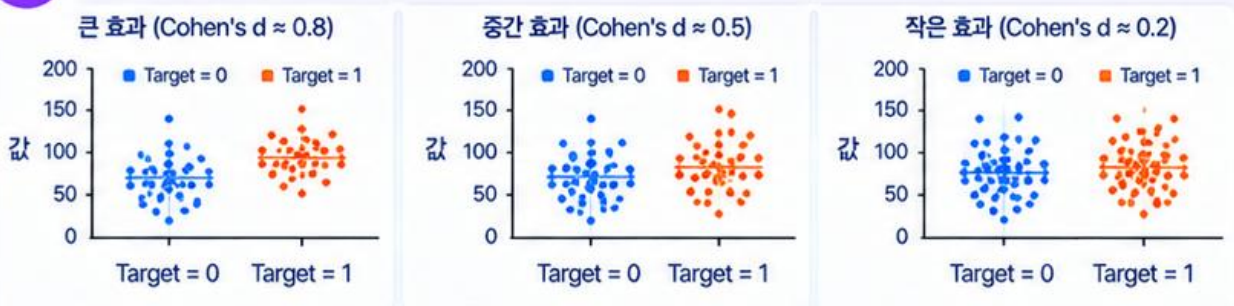
3 표본 수가 크면 p-value는 작아진다

평균이 거의 같은 두 그룹의 예시

표본 수 (n)	평균 차이	p-value	Cohen's d
100	0.5	0.32	0.05
1,000	0.5	0.10	0.05
10,000	0.5	0.01	0.05
100,000	0.5	1e-6	0.05

- 표본 수가 커질수록 아주 작은 차이도 통계적으로 유의해질 수 있습니다.
- 하지만 Cohen's d는 동일하여, 실제 효과 크기는 변하지 않습니다.
- 따라서 p-value만으로 변수의 중요성을 판단하면 잘못된 결론을 내릴 수 있습니다.

4 시각적 예시



5 변수 중요도 판단 시 함께 고려할 것

1. **p-value**
   통계적으로 차이가 있는가?
2. **Effect size (Cohen's d, Cramér's V 등)**
   그 차이가 얼마나 큰가?
3. **Validation performance**
   실제 모델에서 도움이 되는가?
   (예: 단변량 AUC, 교차검증 성능, 모델 중요도)

6 핵심 메시지

- p-value ≠ Feature Importance
  (유의하다고 해서 중요한 변수가 아님)
- p-value와 Effect size를 함께 본다.
- 표본 수가 크면 p-value는 작아질 수 있으므로, 실제 효과 크기와 모델 성능을 함께 고려한다.

2차시
11

통계로 데이터를 진단하는 방법

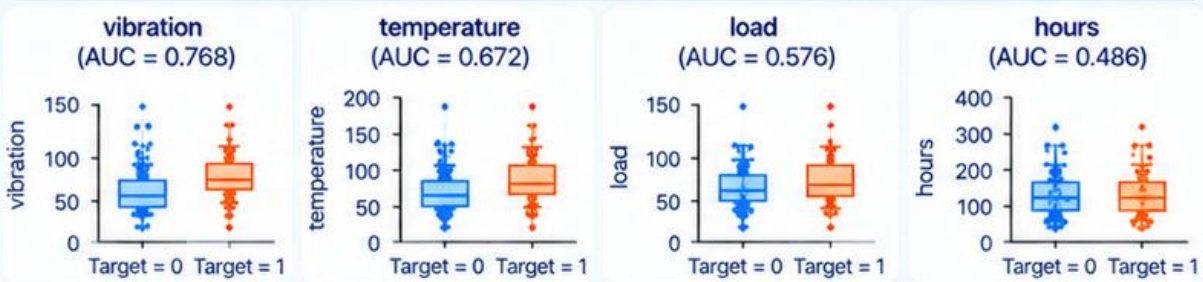
실제 데이터 예시 — 같은 Feature도 문제에 따라 가치가 달라진다

Feature의 중요성은 데이터 자체의 고정된 속성이 아닙니다. 문제(Task)와 Target이 바뀌면, 같은 변수라도 중요도가 달라질 수 있습니다.

1 분류 문제 (Binary Classification)

Target = 0 vs 1 (예: 고장 발생 여부)

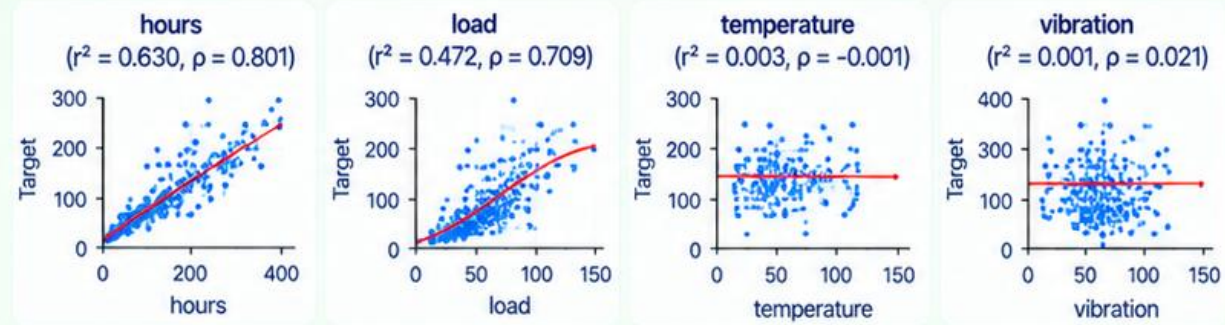
Feature	p-value	Cohen's d	Univ. AUC	해석
vibration	6e-51	1.04	0.768	큰 효과 (매우 중요한 변수)
temperature	2e-22	0.65	0.672	중간 효과 (중요한 변수)
load	4e-05	0.26	0.576	유의하지만 작은 효과
hours	0.46	-0.05	0.486	신호 없음 (중요하지 않음)



2 회귀 문제 (Regression)

연속형 Target (예: 장비 수명, 연속 값 예측)

Feature	Pearson r	r^2 (설명력)	Spearman ρ	해석
hours	0.794	0.630	0.801	가장 강한 예측자 (매우 중요한 변수)
load	0.687	0.472	0.709	중요한 변수
temperature	-0.052	0.003	-0.001	거의 관련 없음
vibration	0.030	0.001	0.021	거의 관련 없음



3 학생에게 던지는 질문

- p-value만 보면 load를 중요한 변수라고 판단했을까요?
- classification에서 신호가 없었던 hours가 regression에서는 왜 가장 강한 예측자가 될까요?

4 핵심 메시지

- p-value는 '차이가 있는가?'를 말할 뿐, 변수의 실제 중요도를 나타내지 않습니다.
- Effect size(Cohen's d, r^2, ρ)와 모델 성능(AUC 등)을 함께 봐야 합니다.
- Feature의 중요성은 고정된 속성이 아닙니다. 문제(Task)와 Target이 바뀌면 중요도도 달라집니다.

5 실무적인 시사점

- 분석 목적에 맞는 통계 지표를 선택합니다.
- p-value + effect size + 모델 성능을 함께 해석합니다.
- 같은 변수가 여러 문제에서 다른 가치를 가질 수 있음을 항상 염두에 둡니다.
- 변수를 삭제하기 전에 도메인 지식과 모델 성능 변화를 함께 고려합니다.

2차시 11

통계로 데이터를 진단하는 방법

다중비교와 다중공선성 — 우연한 유의성과 중복된 정보에 주의하자

변수가 많아질수록 우연히 유의한 변수가 생길 수 있으며, 서로 강하게 상관된 변수는 같은 정보를 제공합니다. 올바른 보정과 해석이 필요합니다.

1 다중비교 문제 (Multiple Comparison Problem)

변수가 많으면 우연히 유의한 결과가 더 많이 나올 수 있습니다.

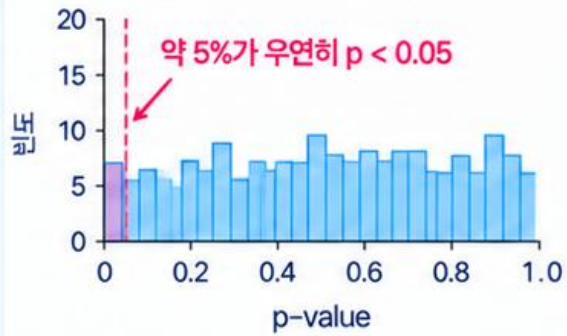
모든 변수가 실제로 무관하다고 가정해도...

- 유의수준 $\alpha = 0.05$를 사용하면
- 100개의 변수를 검정할 때
- 평균적으로 약 5개가
- 우연히 유의하게 나올 수 있습니다.



100개 중 약 5개가 우연히 유의 ($p < 0.05$)

모든 변수가 무관한 경우의 p-value 분포 (예시)



2 다중비교 보정 방법

여러 번의 검정으로 인한 우연한 유의성을 줄이기 위해 p-value를 보정합니다.

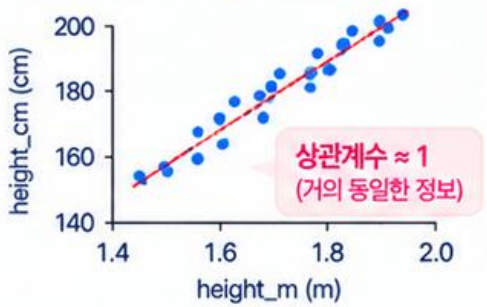
방법	주요 아이디어	특징	적용 상황
Bonferroni	$\alpha' = \alpha / m$ (m: 검정 수)	매우 보수적 (Type I 오류 강하게 제어)	변수 수가 적을 때
Holm	단계적으로 α를 조정	중간 수준의 보정 (Bonferroni보다 덜 보수적)	일반적인 상황
Benjamini–Hochberg (BH)	FDR(False Discovery Rate) 제어	실무에서 많이 사용 (탐색 분석에 적합)	**EDA에서 기본적으로 권장**

! 대회 EDA에서는 보통 **Benjamini–Hochberg 방법**을 사용하고, 이후 교차검증을 2차 관문으로 사용하여 실제 예측 성능에서도 유효한 변수를 확인합니다.

3 다중공선성(Multicollinearity)

서로 강하게 상관된 변수들이 많으면 같은 정보를 중복으로 가지고 있는 것입니다.

예시: height_cm과 height_m



상관행렬 (Correlation Matrix) 예시

	height_cm	height_m	weight	age
height_cm	1.00	1.00	0.45	0.12
height_m	1.00	1.00	0.44	0.11
weight	0.45	0.44	1.00	0.25
age	0.12	0.11	0.25	1.00

1.0 - 0.5 - 0.0

```
# 상관행렬 확인
train[numeric_cols].corr()
```

4 VIF (Variance Inflation Factor)

각 변수가 다른 변수들에 의해 얼마나 설명되는지를 수치로 확인합니다.

$$VIF_j = \frac{1}{1 - R_j^2}$$

R_j^2 : 해당 변수를 나머지 변수들로 회귀했을 때의 결정계수

VIF 값	해석
VIF < 5	낮음 (문제 없음)
5 ≤ VIF ≤ 10	주의 (다중공선성 가능)
VIF > 10	높음 (다중공선성 심함)

5 VIF가 높으면 무조건 삭제할까?

아닙니다.

- 다중공선성은 주로 계수 해석을 어렵게 하는 문제입니다.
- Linear Regression에서 계수가 데이터에 따라 불안정할 수 있습니다.
- 하지만 예측 성능이 반도시 크게 나빠지는 것은 아닙니다.

따라서...

- 모델의 목적에 따라 판단합니다.
- 계수 해석이 중요하면 변수를 줄이거나 정규화(Ridge, Lasso 등)를 고려합니다.
- 예측 성능이 중요하면 굳이 삭제하지 않고 모델의 성능을 확인합니다.

2차시 12

통계로 데이터를 진단하는 방법

Train과 Test는 정말 같은 데이터인가? — Distribution Shift

머신러닝에서는 암묵적으로 $P_{train}(X) \approx P_{test}(X)$를 기대하지만, 실제 대회에서는 시간, 장비, 환경 등의 이유로 분포가 달라질 수 있습니다.

1 Distribution Shift란?

Train과 Test의 데이터 분포가 다른 현상

Train 데이터
- 2024년 데이터
- Device A/B/C
- 특정 지역 / 환경



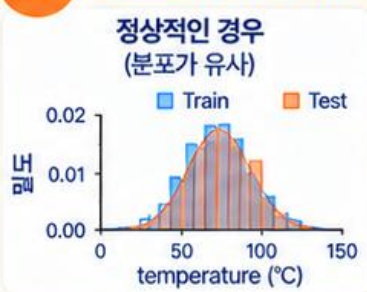
≠

Test 데이터
- 2026년 데이터
- Device D 포함
- 다른 지역 / 환경

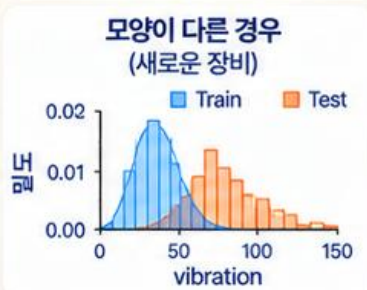


학습 시의 분포와 실제 예측 시의 분포가 다르면 모델의 성능이 크게 떨어질 수 있습니다.

2 분포 차이의 예시









3 KS Test (Kolmogorov-Smirnov Test)

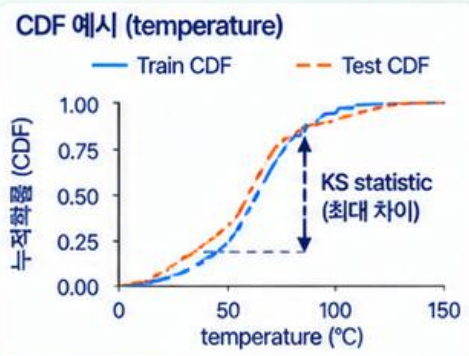
두 연속형 분포의 누적분포함수(CDF) 차이를 측정

방법 개요
- Train과 Test의 CDF의 최대 차이를 측정
- KS statistic과 p-value를 함께 해석
- 큰 데이터에서는 작은 차이도 통계적으로 유의할 수 있으므로 두 값을 함께 봐야 함

$$KS = \max_x \left| F_{train}(x) - F_{test}(x) \right|$$

$F_{train}(x)$: Train의 누적분포함수

$F_{test}(x)$: Test의 누적분포함수



결과 해석 예시

KS statistic	p-value	해석
0.05	0.32	유의한 차이 없음
0.15	0.001	분포 차이 있음
0.30	1e-10	매우 큰 분포 차이

4 PSI (Population Stability Index)

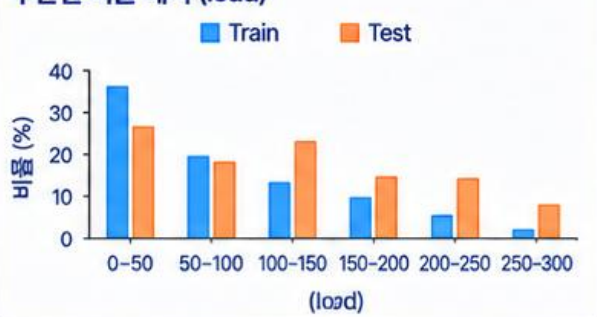
구간별 비율 차이를 이용하여 분포 변화 정도를 측정

계산 방법
1. Train과 Test를 동일한 구간(bin)으로 나눔
2. 각 구간의 비율을 계산 (p_i, q_i)
3. 다음 식으로 PSI 계산

$$PSI = \sum_{i=1}^{k} (p_i - q_i) \times \ln\left(\frac{p_i}{q_i}\right)$$

p_i : Train의 i번째 구간 비율

q_i : Test의 i번째 구간 비율



PSI 해석 기준

PSI < 0.10	Stable (안정)
0.10 ~ 0.25	Warning (주의)
> 0.25	Shift (분포 변화 큼)

해석 시 주의사항
- 구간(bin) 개수는 보통 10개 사용
- 변수의 특성에 맞게 구간을 조정

5 실무적인 접근 방법

1. 주요 변수에 대해 KS test와 PSI를 계산한다.
2. 분포 변화가 큰 변수(PSI > 0.25 등)를 확인한다.
3. 원인을 분석한다.
   - 시간 변화 (2024 → 2026)
   - 새로운 장비 / 센서
   - 지역 / 환경 차이
   - 데이터 수집 정책 변경
4. 필요한 경우 처리한다.
   - 시간 변수 추가
   - 도메인 기반 전처리
   - Train과 유사하게 보정 (Reweighting 등)
   - Shift에 강한 모델 사용

핵심 메시지

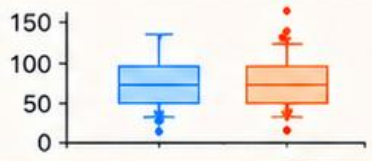
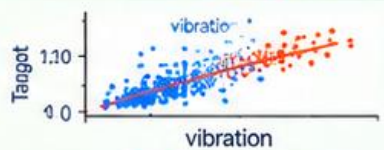
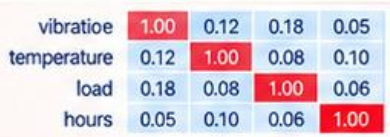
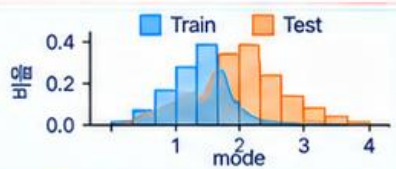
- Train과 Test의 분포가 항상 같다고 가정하면 안 된다.
- KS statistic와 p-value, PSI를 함께 확인한다.
- 분포 변화의 원인을 파악하고 적절한 대응이 필요하다.
- 분포가 다르면 모델 성능이 크게 떨어질 수 있다.

2차시 최종정리

통계로 데이터를 진단하는 방법

EDA는 전처리 의사결정으로 끝나야 한다 — EDA Diagnostic Report

지금까지 수행한 다양한 진단 결과를 하나의 표로 정리하고, 실제 전처리 의사결정으로 연결합니다.

항목	진단 결과 (예시)	관찰 (Observation)	판단 (Judgment)	다음 행동 (Action)
Distribution (분포)	 • skew ≈ 0 • kurtosis ≈ -1.2	• 대칭에 가까운 분포 • 정규분포보다 평평한 형태 (kurtosis < 0)	**대칭, 평평한 분포** 변환이 필요하지 않음	**변환 불필요** • 로그 변환 등은 적용하지 않음 • 현재 분포 유지
Outlier (이상치)	 • temperature 이상치 5건	• IQR, robust z-score 기준에서 이상치 후보 5건 • 물리적으로 가능한 값의 범위	**물리적으로 가능** 오류로 보기 어려움	**유지** • 삭제하지 않음 • 필요 시 모델에서 강건한 방법 (예: Tree 기반 모델, clipping) 고려
Missing (결측치)	 • 결측 비율 3.5 ~ 4.9% • MAR의 강한 증거 없음	• 결측 비율이 낮은 편 • 결측이 다른 관측 변수로 설명된다는 뚜렷한 증거 없음	**MAR 증거 없음** 일반적인 결측 처리로 충분	**median / mode 대체** • 수치형: median 대체 • 범주형: mode 대체 • 필요 시 Missing Indicator 추가
Association (Feature–Target)	 • vibration p = 6e-51 • Cohen's d = 1.04 • Univ. AUC = 0.768	• Target과 강한 관계 • 다른 변수보다 높은 예측력 확인	**주요 예측 변수** 모델에 중요할 가능성이 큼	**유지** • 핵심 변수로 모델링에 사용 • 추가 파생변수 생성도 고려
Collinearity (다중공선성)	 • 모든 변수 VIF ≈ 1 • 상관계수 낮음	• 강한 상관관계가 없음 • VIF 값이 모두 낮음	**문제 없음** 다중공선성 우려 없음	**조치 없음** • 변수 제거 불필요 • 현재 변수 구성 유지
Shift (Train–Test 분포 차이)	 • mode의 PSI = 0.359 • Train에는 없던 새로운 범주 존재	• Train과 Test의 분포가 다름 • Test에만 존재하는 새로운 범주 확인	**새 범주 존재** 분포 변화(Shift) 발생	**unknown 처리** • Train에 없는 범주는 'unknown' 카테고리로 처리 • 모델이 새로운 범주에 대응할 수 있도록 조정

핵심 메시지

- EDA는 단순히 데이터의 특성을 확인하는 것이 아니라, 전처리 의사결정으로 연결되어야 합니다.
- p-value만 보지 말고, 효과 크기와 실제 문제 맥락을 함께 고려합니다.
- 데이터는 항상 같지 않을 수 있으며(Distribution Shift), 이를 고려한 적절한 대응이 필요합니다.

3차시

전처리와 Validation 설계

좋은 모델보다 먼저 좋은 검증셋을 만든다

핵심 질문

"내 Validation Score를 믿어도 되는가?"

1 Train / Validation / Test

각 데이터의 역할을 명확히 구분합니다.



- Train : 모델을 학습하는 데이터
- Validation : 하이퍼파라미터 튜닝, 모델 선택에 사용
- Test : 최종 평가용 (모든 과정을 마친 후 1회만 사용)

2 데이터 분할 방법

문제의 특성에 맞는 분할 방법을 선택합니다.

Random Split

데이터를 무작위로 분할합니다.

```
train_test_split(
  test_size=0.2,
  random_state=42
)
```



Stratified Split

분류 문제에서 클래스 비율을 유지하여 분할합니다.



Group Split

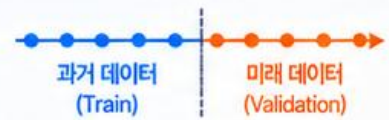
같은 그룹(사람, 장비 등)이 여러 번 측정된 경우 사용합니다.



- GroupShuffleSplit : 단일 분할
- GroupKFold : 교차검증

Time Split

시간 순서를 지켜서 분할합니다.



- 시계열 데이터, 기간에 따라 분포가 달라지는 경우
- 과거 → Train, 미래 → Validation

3 Data Leakage (데이터 누수)

Train이 아닌 정보가 학습에 사용되면 비현실적으로 좋은 성능이 나옵니다.

✕ 잘못된 방법 (Leakage 발생)



- 전체 데이터의 정보를 미리 사용하여 Validation 성능이 실재보다 좋게 나옴
- 실제 서비스에서는 성능이 크게 떨어질 수 있음

✓ 올바른 방법 (Leakage 방지)



- Train 데이터에서만 전처리 기준을 계산
- Validation/Test에는 동일한 기준만 적용
- 현실적인 성능 평가가 가능

4 분할 방법 선택 가이드

데이터 특성	권장 방법	사용 예시
일반적인 데이터	Random Split	대부분의 회귀/분류 문제
클래스 불균형 (분류)	Stratified Split	불량 탐지, 질병 분류 등
같은 그룹의 반복 측정	Group Split	같은 환자, 같은 장비, 같은 사용자
시간 흐름이 있는 데이터	Time Split	매출 예측, 센서 데이터, 로그 데이터 등

5 실습 예시 (scikit-learn)

```
# 1) Random Split
from sklearn.model_selection import train_test_split
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42)
)

# 2) Stratified Split (분류)
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
)
# 3) Group Split
from sklearn.model_selection import GroupShuffleSplit

# 4) Time Split
# 데이터의 시간 순서를 기준으로 직접 분할
```

6 핵심 메시지

- 좋은 모델보다 먼저 좋은 검증셋을 만든다.
- 문제의 특성에 맞는 분할 방법을 선택한다.
- 전처리는 반드시 Train 데이터에서만 fit하고, Validation/Test에는 같은 기준만 적용한다.
- Group, Time과 같은 구조를 무시하면 데이터 누수가 발생할 수 있다.
- 내 Validation Score를 믿을 수 있는지 항상 의심하고 확인한다.

실전 1
Classification

Accuracy만 보지 않는 분류 모델링

문제의 특성과 비용을 고려하여 적절한 모델, 평가 지표, 임계값을 선택한다.

1 분류 모델 개요

여러 모델을 비교하여 데이터에 적합한 모델을 선택합니다.

모델	설명
DummyClassifier	가장 단순한 기준 모델 (성능의 하한선)
Logistic Regression	해석이 쉬운 선형 모델 (기본 모델)
Random Forest	비선형 관계를 잘 잡는 트리 기반 모델
XGBoost	성능이 좋은 부스팅 모델 (많이 사용)
LightGBM	빠르고 성능이 좋은 부스팅 모델
CatBoost	범주형 변수 처리에 강한 부스팅 모델

- 대회 환경에서 설치 가능한 라이브러리인지 먼저 확인합니다.
- Dummy 모델과 비교하여 실제로 성능이 향상되었는지 확인합니다.

2 주요 평가 지표

여러 지표를 함께 봐야 모델을 올바르게 평가할 수 있습니다.

Accuracy (정확도)

전체 중 맞춘 비율

$$\frac{(TP + TN)}{(TP + TN + FP + FN)}$$

Precision (정밀도)

Positive로 예측한 것 중 실제로 Positive인 비율

$$\frac{TP}{(TP + FP)}$$

Recall (재현율)

실제 Positive 중 올바르게 예측한 비율

$$\frac{TP}{(TP + FN)}$$

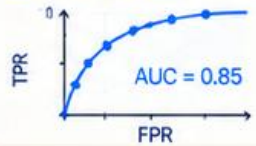
F1 Score

Precision과 Recall의 조화평균

$$F1 = 2\frac{2\times(\text{Precision} \times \text{Recall})}{(\text{Precision} + \text{Recall})}$$

ROC-AUC

모든 임계값에서의 분류 성능 (클래스 구분 능력)



Confusion Matrix (혼동행렬)

실제 \ 예측	Negative	Positive
Negative	TN	FP
Positive	FN	TP

3 Confusion Matrix와 문제의 비용

문제 도메인에 따라 FP와 FN의 비용이 다를 수 있습니다.

		예측 결과: 정상 (Negative)	예측 결과: 양성 (Positive)
실제 결과	정상 (Negative)	TN (정상 예측)	FP (오탐)
실제 결과	양성 (Positive)	FN (놓침)	TP (정탐)

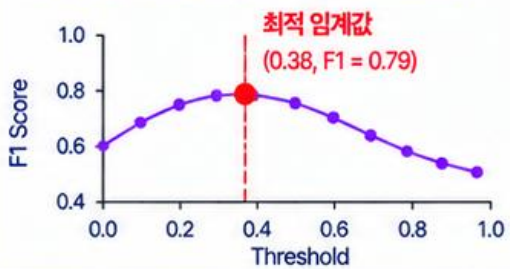
의료 분야 예시 (암 탐지)

- False Negative (실제 암을 정상으로 예측) 의 비용이 False Positive보다 훨씬 클 수 있습니다.
- 문제의 목적에 맞게 중요한 지표(예: Recall)를 선택해야 합니다.

4 임계값(Threshold) 최적화

기본값 0.5가 항상 최선은 아닙니다.

임계값에 따른 성능 변화 예시 (F1 Score)



Threshold	F1 Score
0.3	0.76
0.4	**0.79**
0.5	0.75
0.6	0.71
0.7	0.63

- 확률값(probability)에 임계값을 적용하여 클래스 라벨을 결정합니다.
- 임계값은 validation 데이터에서 다양한 값을 비교하여 선택합니다.

5 임계값 선택 시 주의사항

같은 데이터에서 모델 학습, 튜닝, 임계값 선택을 모두 하면 overfitting이 발생할 수 있습니다.

데이터 분할 예시 (fit / tune / check 구조)

Train (모델 학습)	Validation (Tune) (모델 파라미터 튜닝 + 임계값 선택)	Check (독립 검증)

모델 학습 (파라미터 학습) → 최적 임계값 선택 (여러 threshold 비교) → 독립 데이터에서 최종 성능 확인

- 임계값까지 같은 validation에서 반복 선택하면 과적합될 수 있습니다.
- 가능하면 tune과 check 데이터를 분리하여, check 데이터에서 최종 성능을 검증합니다.

6 실습 결과 예시

여러 모델과 임계값을 비교하여 최적의 성능을 찾습니다.

모델	Threshold	Accuracy	Precision	Recall	F1	ROC-AUC
Dummy	-	0.50	0.50	0.50	0.50	0.50
Logistic Regression	0.5	0.82	0.78	0.75	0.76	0.86
Random Forest	0.5	0.85	0.82	0.78	0.80	0.89
XGBoost	0.5	0.86	0.83	0.79	0.81	0.90
XGBoost (최적 임계값)	**0.38**	0.84	0.76	**0.83**	**0.79**	**0.90**

- 모델별 성능은 데이터에 따라 달라질 수 있습니다.
- 임계값 조정을 통해 F1, Recall 등 원하는 지표를 향상시킬 수 있습니다.

XGBoost 한눈에 보기

XGBoost

eXtreme Gradient Boosting

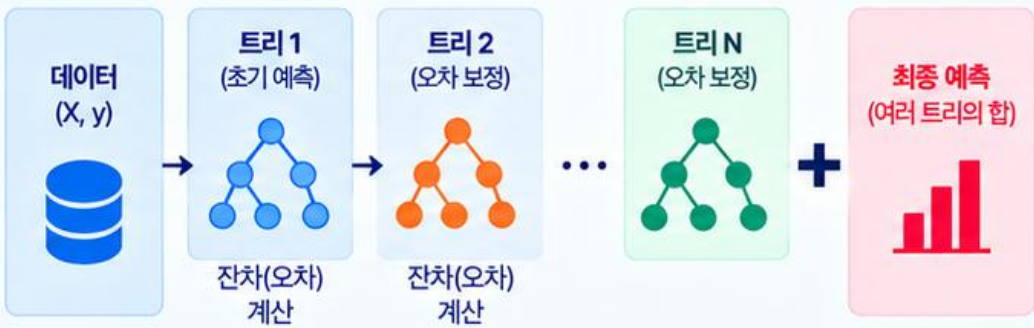
결정트리(Decision Tree)를 기반으로 한 강력한 앙상블 모델로, 대부분의 정형 데이터 대회에서 높은 성능을 보이는 알고리즘입니다.

핵심 포인트

- 빠르고 높은 예측 성능
- 과적합을 줄이기 위한 다양한 정규화 기법
- 정형 데이터 대회에서 가장 널리 사용되는 모델 중 하나

1 XGBoost란?

여러 개의 약한 결정트리를 순차적으로 학습하여
이전 트리의 오차를 보정해 나가는 그래디언트 부스팅 알고리즘입니다.



2 어떻게 학습하나?

손실함수를 최소화하기 위해 그래디언트와 헤시안(2차 미분) 정보를 사용하여 트리를 순차적으로 추가합니다.

$$Obj(\theta) = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k)$$

첫 번째 항
예측 오차
(손실함수, Loss)

두 번째 항
모델 복잡도에 대한 페널티
(정규화, Regularization)

오차를 줄이면서도
너무 복잡한 모델이 되지 않도록 균형을 맞춥니다.

3 주요 특징

높은 예측 성능
대부분의 정형 데이터에서 좋은 성능을 보임

빠른 학습 속도
병렬 처리와 효율적인 알고리즘으로 빠르게 학습

과적합 방지 기능
다양한 정규화(regularization) 파라미터 제공

결측치 처리
별도의 전처리 없이 결측치를 자동으로 처리

다양한 목적 함수
분류, 회귀, 랭킹 등 다양한 문제에 적용 가능

4 XGBoost의 장점과 단점

장점

- 높은 예측 성능 (대회에서 강력한 베이스라인)
- 과적합을 줄이는 다양한 정규화 기법
- 결측치 처리, 이상치에 비교적 강건함
- 대용량 데이터도 빠르게 학습 가능
- 분류, 회귀, 랭킹 등 다양한 문제에 적용 가능

단점

- 하이퍼파라미터가 많아 튜닝이 필요함
- 해석이 비교적 어렵고 블랙박스 성향이 강함
- 범주형 변수를 그대로 사용하려면 추가 설정이 필요할 수 있음
- 데이터가 매우 작을 경우 과적합 될 수 있음
- 학습 시간이 다른 단순 모델보다 길 수 있음

5 주요 하이퍼파라미터

파라미터	설명	일반적인 값 범위
n_estimators	트리 개수	100 ~ 1000
learning_rate (eta)	각 트리의 기여도 (적을수록 천천히 학습)	0.01 ~ 0.3
max_depth	트리의 최대 깊이 (모델 복잡도)	3 ~ 10
subsample	학습할 데이터의 비율	0.5 ~ 1.0
colsample_bytree	각 트리에서 사용할 특성의 비율	0.5 ~ 1.0
reg_alpha (L1)	L1 정규화 (특성 선택 효과)	0 ~ 10
reg_lambda (L2)	L2 정규화 (과적합 방지)	0 ~ 10

6 간단한 사용 예시 (Python)

```python
from xgboost import XGBClassifier

model = XGBClassifier(
    n_estimators=500,
    learning_rate=0.05,
    max_depth=6,,
    subsample=0.8,
    colsample_bytree=0.8,
    reg_alpha=0.1,
    reg_lambda=1.0,
    random_state=42
)

model.fit(X_train, y_train)
pred = model.predict(X_valid)
prob = model.predict_proba(X_valid)[:, 1]
)
```

7 주요 활용 분야

분류 문제
고객 이탈 예측, 이상 탐지, 질병 진단 등

회귀 문제
매출 예측, 가격 예측, 수요 예측 등

랭킹 문제
검색 결과 순위, 추천 시스템

데이터 분석 대회
Kaggle 등 대부분의 정형 데이터 대회에서 널리 사용

LightGBM 한눈에 보기

LightGBM

Light Gradient Boosting Machine

Microsoft에서 개발한 빠르고 효율적인 그래디언트 부스팅 의사결정트리(GBDT) 알고리즘으로, 대규모 데이터에서 높은 성능과 속도를 제공합니다.

핵심 포인트

- 대규모 데이터에서도 빠른 학습 속도
- 메모리 사용량이 적고 효율적
- 대회(캐글)에서 널리 사용되는 강력한 베이스라인 모델

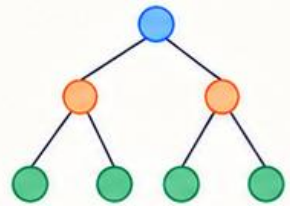
1 LightGBM이란?

여러 개의 의사결정트리를 순차적으로 학습하는 그래디언트 부스팅 모델로, 기존 트리 기반 모델을 개선하여 더 빠르고 효율적인 학습이 가능합니다.

리프 중심 분할 (Leaf-wise split)	손실을 많이 줄이는 리프부터 분할하여 더 적은 트리로 높은 성능 달성
히스토그램 기반 (Histogram-based)	연속형 변수를 구간으로 나누어 계산 속도와 메모리 효율성 향상
GOSS (Gradient-based One-Side Sampling)	중요한 데이터만 샘플링하여 빠른 학습 속도 구현
EFB (Exclusive Feature Bundling)	서로 동시에 0이 되지 않는 변수를 하나로 묶어 차원 축소 (메모리 절약)

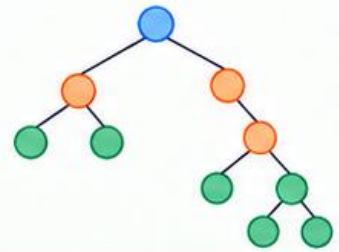
2 트리 분할 방식 비교

Level-wise (일반적인 방법)



- 한 레벨을 순차적으로 분할
- 균형 잡힌 트리 구조
- 분할 속도가 상대적으로 느림
- 더 많은 트리가 필요할 수 있음

Leaf-wise (LightGBM)



- 손실을 가장 많이 줄이는 리프부터 분할
- 더 적은 트리로 높은 성능
- 학습 속도가 빠름
- 깊은 트리가 생길 수 있어 과적합 주의

(→ max_depth, min_data_in_leaf 등으로 제어)

3 주요 특징 및 장점

- **빠른 학습 속도**
  히스토그램 기반 알고리즘과 GOSS로 대규모 데이터에서도 빠르게 학습
- **낮은 메모리 사용량**
  EFB를 통해 여러 변수를 하나로 묶어 메모리 효율성 향상
- **높은 예측 성능**
  적은 트리로도 높은 성능을 보여 대회에서 널리 사용
- **다양한 데이터 타입 지원**
  수치형, 범주형(자동 처리 가능), 결측치 처리 등 다양한 상황에 강함

4 주의사항 (단점)

- Leaf-wise 방식으로 인해 깊은 트리가 생성되어 과적합될 수 있음
- 적절한 하이퍼파라미터 튜닝이 필수 (특히 max_depth, min_data_in_leaf)
- 작은 데이터셋에서는 다른 모델(XGBoost 등)보다 성능이 떨어질 수 있음

5 주요 하이퍼파라미터

파라미터	설명	일반적인 값 범위
n_estimators	트리 개수	100 ~ 5000
learning_rate	학습률 (작을수록 천천히 학습)	0.01 ~ 0.3
num_leaves	리프 노드의 최대 개수 (모델 복잡도)	20 ~ 300
max_depth	트리의 최대 깊이 (-1이면 제한 없음)	-1 ~ 20
min_data_in_leaf	리프 노드에 필요한 최소 샘플 수 (과적합 방지)	10 ~ 100
feature_fraction	사용할 feature의 비율	0.5 ~ 1.0
bagging_fraction	데이터 샘플링 비율	0.5 ~ 1.0
lambda_l1 / lambda_l2	L1, L2 정규화 (과적합 방지)	0 ~ 10

6 Python 사용 예시

```python
import lightgbm as lgb
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

# 1. 데이터 분할
X_train, X_valid, y_train, y_valid = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# 3. 예측 및 평가
model = lgb.LGBMClassifier(
    n_estimators=1000,
    learning_rate=0.05,
    num_leaves=31,
    max_depth=-1,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42
)
model.fit(X_train, y_train,
    eval_set=[(X_valid, y_valid)],
    eval_metric='auc',
    early_stopping_rounds=100,
    verbose=False)
```

7 적용 분야 및 활용 팁

적용 분야

- 캐글 등 데이터 분석 대회 (Tabular 데이터)
- 신용평가, 고객 이탈 예측, 이상 탐지
- 추천 시스템, 의료, 마케팅 등 다양한 실무 분야

활용 팁

- 범주형 변수는 category 타입으로 지정 (자동 처리 가능)
- early_stopping 사용으로 최적의 트리 개수 찾기
- num_leaves, min_data_in_leaf로 과적합 조절
- 교차검증과 하이퍼파라미터 튜닝이 성능 향상에 중요

언제 사용하면 좋을까?

- 데이터가 크고 변수(컬럼)가 많은 경우
- 빠른 학습 속도와 높은 성능이 모두 필요한 경우
- 범주형 변수가 많은 데이터

대안 모델과 비교

- XGBoost: 더 많은 하이퍼파라미터, 약간 느림
- CatBoost: 범주형 변수 처리에 특화
- LightGBM: 빠르고 효율적인 성능, 대회에서 널리 사용

CatBoost 한눈에 보기

CatBoost

범주형 변수에 강한 그래디언트 부스팅 모델

Yandex에서 개발한 그래디언트 부스팅 알고리즘으로, 범주형 변수를 별도의 전처리 없이 효율적으로 처리할 수 있습니다.

핵심 포인트

- 범주형 변수를 별도의 인코딩 없이 바로 사용
- 과적합을 줄이는 특화된 알고리즘 (Ordered Boosting)
- 대회(캐글)에서 안정적이고 높은 성능을 보이는 모델

1 CatBoost란?

그래디언트 부스팅(GBDT) 기반 모델로,
범주형 변수 처리와 과적합 방지에 특화되어
대회 데이터에서 좋은 성능을 보입니다.

범주형 변수 자동 처리
One-Hot Encoding 없이
범주형 변수를 바로 사용 가능

과적합 방지 (Ordered Boosting)
타깃 누수를 방지하는 특수한 학습 방식

안정적인 성능
작은 데이터셋에서도 좋은 성능,
파라미터 튜닝이 비교적 쉬움

다양한 작업 지원
분류, 회귀, 랭킹 등 여러 문제에 적용 가능

2 범주형 변수 처리 방식

CatBoost는 Ordered Target Encoding이라는 방법으로
범주형 변수를 안전하게 처리합니다.

일반적인 Target Encoding
(타깃 누수 위험)

데이터	범주형 변수 (지역)	타깃 (구매여부)	인코딩 값 (평균)
1	A	1	0.67
2	A	0	0.67
3	A	1	0.67
4	B	0	0.25
5	B	0	0.25
6	B	1	0.25
...	...	...	...

전체 데이터를 사용하여 평균을 계산하므로
타깃 정보가 미리 반영되어 누수가 발생할 수 있음

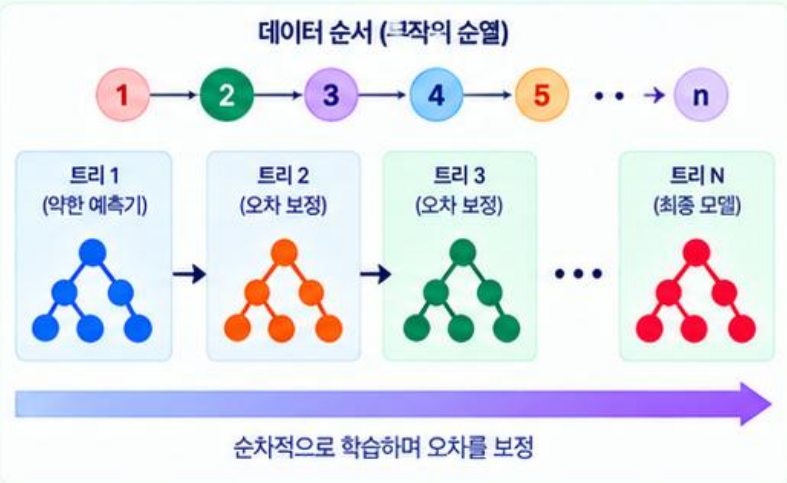
CatBoost의 Ordered Target Encoding
(타깃 누수 방지)

데이터	범주형 변수 (지역)	타깃 (구매여부)	인코딩 값 (이전 데이터만 사용)
1	A	1	0.50 (사전값)
2	A	0	1.00 (이전 1개)
3	A	1	0.50 (이전 2개)
4	B	0	0.50 (사전값)
5	B	0	0.00 (이전 1개)
6	B	1	0.00 (이전 2개)
...	...	...	...

각 샘플보다 이전 데이터만 사용하여 인코딩하므로
타깃 누수를 방지하고 과적합을 줄임

3 학습 과정 (Ordered Boosting)

데이터를 무작위 순서로 나열하고, 이전 데이터만 사용하여
타깃 통계를 계산하면서 트리를 순차적으로 학습합니다.



4 주요 하이퍼파라미터

파라미터	설명	일반적인 값 범위
iterations	트리 개수	500 ~ 5000
learning_rate	학습률 (작을수록 천천히 학습)	0.01 ~ 0.3
depth	트리의 최대 깊이	4 ~ 10
l2_leaf_reg	L2 정규화 (과적합 방지)	1 ~ 10
bagging_temperature	부트스트랩 샘플링 강도	0 ~ 1
random_strength	분할 시 랜덤성 조절	0 ~ 1
early_stopping_rounds	조기 종료 기준	50 ~ 200
task_type	실행 환경 (CPU/GPU)	'CPU' 또는 'GPU'
cat_features	범주형 변수 컬럼 인덱스/이름	예: [0, 3, 5] 또는 ['region', 'device']

5 Python 사용 예시 (분류 문제)

```python
# 1. 라이브러리 불러오기
from catboost import CatBoostClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, roc_auc_score

# 2. 데이터 분할
X_train, X_valid, y_train, y_valid = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 3. 모델 학습 (범주형 변수 지정)
model = CatBoostClassifier(
    iterations=1000,
    learning_rate=0.05,
    depth=6,
    l2_leaf_reg=3,
    random_state=42,
    cat_features=['region', 'device'] # 범주형 변수 이름 또는 인덱스
)
model.fit(X_train, y_train,
          eval_set=(X_valid, y_valid),
          early_stopping_rounds=100,
          verbose=100)

# 4. 예측 및 평가
pred = model.predict(X_valid)
prob = model.predict_proba(X_valid)[:, 1]

print(classification_report(y_valid, pred))
print('ROC-AUC:', roc_auc_score(y_valid, prob))
```

6 CatBoost의 장단점과 활용 팁

장점

- 범주형 변수 전처리 불필요
  (One-Hot Encoding 불필요)
- 타깃 누수 방지로 안정적인 일반화 성능
- 작은 데이터셋에서도 우수한 성능
- 하이퍼파라미터 튜닝이 비교적 쉬움
- 결측치 처리 기능 내장
- GPU 지원으로 빠른 학습 가능
- 대회(캐글)에서 높은 성능을 자주 보임

단점

- XGBoost/LightGBM보다
  학습 속도가 느릴 수 있음
- 메모리 사용량이 상대적으로 큼
- 매우 큰 데이터셋에서는
  다른 모델이 더 효율적일 수 있음

활용 팁

- 범주형 변수는 cat_features로 지정하기
- early_stopping으로 과적합 방지하기
- 대회 환경에서 다른 모델(XGBoost, LightGBM)과 함께 비교해보기

5차시
Regression

Regression + Feature Engineering + Error Analysis

모델을 바꾸기 전에 오차를 분석한다

핵심 포인트
- MAE와 RMSE의 의미와 차이를 이해한다.
- 가설 기반으로 Feature를 생성한다.
- 모델을 바꾸기 전에 반드시 오차를 분석한다.
- 오차 패턴을 통해 다음 개선 방향을 찾는다.

1 회귀 문제와 평가 지표

연속형 값을 예측하는 문제에서 주로 사용하는 평가 지표입니다.

MAE (Mean Absolute Error)

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

- 절대 오차의 평균
- 오차 크기에 선형적으로 반응
- 이상치에 상대적으로 덜 민감
- 해석이 직관적 (단위가 원래 단위와 같음)

RMSE (Root Mean Square Error)

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

- 제곱 오차의 평균의 제곱근
- **큰 오차에 더 민감**
- 이상치가 있을 때 값이 크게 증가
- 오차를 제곱해서 계산하므로 큰 오차를 더 강하게 반영

! 질문: 큰 오차에 더 민감한 지표는? → 정답: RMSE

2 사용할 모델

먼저 단순한 기준 모델부터 시작하여 점진적으로 더 복잡한 모델을 비교합니다.

Mean Baseline (평균 예측)

$$\hat{y} = \bar{y}_{train}$$

- 가장 단순한 기준 모델
- 항상 같은 값 예측
- 반드시 비교 대상으로 사용 (성능 하한선)

Ridge Regression (선형 모델)

$$\min \sum (y_i - \hat{y}_i)^2 + \alpha\|w\|^2$$

- 선형 관계 모델링
- L2 정규화로 과적합 방지
- 해석이 용이
- 기본 성능이 좋은 강력한 베이스라인

Random Forest Regressor (트리 기반 모델)



- 비선형 관계 학습 가능
- 다양한 변수 중요도 제공
- 하이퍼파라미터 튜닝 필요
- 일반적으로 강한 성능을 보임

3 Feature Engineering

도메인 지식을 바탕으로 가설을 세우고, 의미 있는 변수를 생성합니다.

예시

기준 변수	생성할 특징(Feature)
load	load² (비선형 효과)
	temperature × load (상호작용)
temperature	고온/저온 구간 구분
	온도 구간별 통계량
hour	시간 주기성 (sin, cos)
	day / night (범주형)

Feature 생성 과정

1. **가설 수립** (도메인 지식, 데이터 탐색)
2. **Feature 생성** (수치형 변환, 상호작용 등)
3. **같은 CV로 성능 비교** (추가한 Feature의 효과 확인)

중요: Feature를 무작정 많이 만들지 말고, 가설을 먼저 세운다.

4 모델 성능 비교 예시

여러 모델의 성능을 MAE와 RMSE로 비교합니다.

모델	MAE ↓	RMSE ↓
Mean Baseline	12.43	18.21
Ridge Regression	8.67	13.02
Random Forest	6.21	9.38
XGBoost (선택)	5.98	9.12
LightGBM (선택)	5.85	8.91
CatBoost (선택)	5.92	9.05

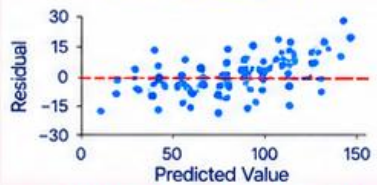
5 Residual(잔차) 분석

모델이 어떤 상황에서 잘못 예측하는지 확인하여, 다음 개선 방향을 찾습니다.

$$e_i = y_i - \hat{y}_i$$

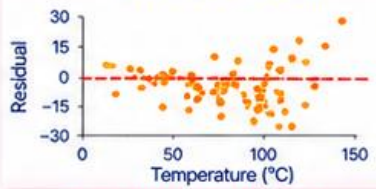
- $e_i > 0$: 실제값이 예측값보다 큼 (과소예측)
- $e_i < 0$: 실제값이 예측값보다 작음 (과대예측)

1 Residual vs Prediction (예측값에 따른 잔차)



예측값이 커질수록 잔차가 커지는 패턴 → 큰 값에서 성능 저하

2 Residual vs Feature (특정 변수에 따른 잔차)



고온 구간에서 잔차가 증가 → temperature 상호작용 검토

3 Large Error Samples (큰 오차 사례 분석)

index	실제값	예측값	잔차	주요 특징
1023	180	95	85	고온, 고부하
4501	20	110	-90	저온, 야간
6720	155	70	85	고온, 장시간
8901	5	60	-55	저부하, 이상치

절대 오차가 큰 샘플의 특징을 확인하여 데이터 문제, 추가 Feature 가능성 탐색

6 오차 분석을 통한 개선 예시

잔차 분석 결과를 바탕으로 다음과 같은 개선을 고려할 수 있습니다.

관찰된 패턴	개선 방향
고온 구간에서 오차 증가	temperature × load 상호작용 변수 추가
큰 값에서 과소예측	비선형 변환 (log, 제곱) 트리 기반 모델 사용
특정 시간대에서 오차 증가	시간 주기성 변수 추가 (day/night, sin/cos)
특정 장비에서 오차가 큼	장비별 구분 변수 추가 (Group 고려)

6차시
실전 전략

성능 개선과 90분 실전 전략

"모델 탐색"이 아니라 "실험 관리"

좋은 성능은 우연이 아니라, 정확한 검증과 체계적인 실험 관리에서 나옵니다.

핵심 메시지

- 모델을 여러 개 돌리기보다, 실험을 체계적으로 관리한다.
- 작은 점수 차이를 바로 개선으로 인정하지 않는다.
- 대회는 제한된 시간(90분) 안에서 우선순위가 중요하다.
- 안정적인 검증과 기본기가 가장 큰 실능 향상으로 이어진다.

1 성능 개선 우선순위

모델을 바꾸기 전에, 먼저 이 순서로 점검합니다.

1	Validation 확인	분할 방법이 문제 없는가? Score를 믿을 수 있는가?
2	Leakage 확인	타겟 정보가 새어 들어가지 않는가?
3	Preprocessing	결측치, 이상치, 스케일링, 인코딩 등
4	Feature	가설 기반의 새로운 변수 생성
5	Model	문제에 맞는 모델 선택
6	Hyperparameter	작은 범위에서 제한적으로 튜닝

2 많은 초보자의 잘못된 순서

Validation 설계가 잘못되어 있으면,
아무리 좋은 모델과 튜닝을 해도 의미가 없습니다.

1. GridSearch (하이파파라미터 튜닝)
2. XGBoost (다른 모델 시도)
3. RandomForest (또 다른 모델 시도)
4. CatBoost (더 좋은 모델 시도)

검증이 잘못되면
모든 시도는 의미가 없습니다.

3 작은 튜닝 전략

대회에서는 큰 범위의 탐색보다
작은 범위에서 빠르게 실험하는 것이 효과적입니다.

파라미터	의미	탐색 범위 예시
max_depth	트리 최대 깊이 (복잡도 조절)	3 / 5 / 7
n_estimators	트리 개수	100 / 300
learning_rate	학습률	0.01 / 0.05 / 0.1
subsample	학습 대이터 샘플 비율	0.6 / 0.8 / 1.0
colsample_bytree	변수 샘플 비율	0.6 / 0.8 / 1.0

- 너무 많은 조합을 탐색하지 않는다.
- 문제에 중요한 파라미터만 선택한다.
- 빠르게 여러 조합을 실험하고 결과를 비교한다.

4 실험 관리 (Experiment Log)

모든 실험을 기록하고 비교해야
어떤 변화가 효과적인지 알 수 있습니다.

Exp	Model	Feature	CV	Score
01	Logistic	Basic	Stratified	0.77
02	Logistic	+ feature1	Stratified	0.78
03	RF	Basic	Stratified	0.75
04	RF	+ feature1	Stratified	0.76
05	XGBoost	Basic	KFold	0.80
06	XGBoost	+ feature1	KFold	0.81

- 데이터 전처리, 사용한 Feature, 모델, 파라미터,
- CV 방법, 점수를 반드시 기록합니다.
- 나중에 성능이 좋을 조합을 쉽게 재현할 수 있습니다.

5 점수 차이가 정말 개선인가?

0.01 차이만으로 개선되었다고 말할 수 있을까요?
분할 방법이나 seed에 따라 점수가 달라질 수 있습니다.

Seed	CV Score (F1)
1	0.782
2	0.776
3	0.791
4	0.783
5	0.779
평균 ± 표준편차	0.782 ± 0.006

모델 A : 0.78 → 모델 B : 0.78

차이 = 0.01

0.01은
실제 개선이라고 볼 수 있을까?

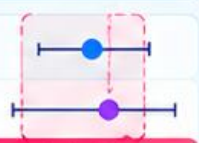
CV의 변동성(표준편차, 신뢰구간)을
함께 확인해야 합니다.

6 Confidence Interval (신뢰구간)

여러 번의 검증 결과로 평균과 신뢰구간을 계산하여,
모델 간 차이가 통계적으로 의미 있는지 판단합니다.

$$\text{평균} \pm 1.96 \times \left(\frac{\text{표준편차}}{\sqrt{n}}\right) \quad (n: \text{반복 횟수})$$

모델 A	0.782 ± 0.012	(0.770 ~ 0.794)
모델 B	0.790 ± 0.013	(0.764 ~ 0.816)



신뢰구간이 겹치면
유의한 차이가 아닐 수 있습니다.

7 90분 실전 전략 (예시)

대회 시간 안에서 효율적으로 진행하는 전략입니다.

0~10분	데이터 이해, EDA 요약
10~20분	Validation 전략 설계 (분할 방법 결정)
20~40분	기본 모델 1~2개 + 기본 전처리
40~60분	Feature 추가 / 모델 개선 (작은 튜닝)
60~80분	추가 실험 및 결과 비교 (Experiment Log 작성)
80~90분	최종 모델 선정 및 제출 파일 생성

실전 팁

- 처음부터 복잡한 모델을 사용하지 않는다.
- 반드시 Validation을 먼저 점검한다.
- 실험 로그를 간단하게라도 기록한다.
- 작은 점수 차이에 과도하게 집착하지 않는다.
- 시간을 나누어 우선순위대로 진행한다.

Thank you.

ryp1662@gmail.com

Youngpyo Ryu. All rights reserved.
Unauthorized reproduction or distribution is prohibited.